Project Title

“Insurance Claim Fraud Detection – AI-Powered Monitoring & Alert System”

Industry

Insurance / FinTech / Risk & Compliance

Target Users

- Insurance Policy Holders
- Claims Processing Teams
- Fraud Detection & Compliance Officers
- Insurance Agents & Brokers
- Regulatory Authorities

Problem Statement

Insurance companies face increasing losses due to **fraudulent claims** such as staged accidents, falsified medical reports, inflated damages, and duplicate claims.

- Manual verification is slow and inefficient.
- Fraudulent claims often go unnoticed until after payouts, causing financial loss.
- Lack of real-time fraud detection increases risk exposure.
- Compliance officers need accurate data to ensure **regulatory adherence**.

Proposed Salesforce-Based Solution

A **Salesforce AI-powered fraud detection CRM** that analyzes claims, detects anomalies, and alerts fraud investigators in real time.

Features:

1. Claim Lifecycle Management

- Custom objects for **Policy**, **Claim**, and **Customer Profile**.
- Track claims from submission → review → approval/rejection.

2. AI-Powered Fraud Detection (Einstein AI/ML)

- Detect unusual claim patterns (e.g., same claimant multiple times, mismatched documents, exaggerated expenses).
- Assign a **fraud risk score** to each claim.
- Prioritize suspicious claims for manual review.

3. Document Verification

- Integrate OCR/AI tools to scan uploaded invoices, medical reports, repair bills.
- Flag forged or duplicate documents.

4. Fraud Alerts & Case Creation

- Automatic alert when claim exceeds threshold risk score.
- Salesforce **Case Management** auto-creates an investigation case.
- Assigns fraud officer for resolution.

5. Customer & Agent Profiling

- Track agent performance and claim submission patterns.
- Identify suspicious agents or policyholders with repeat fraudulent activity.

6. Dashboards & Reporting

- Fraud Detection Rate (%)
- Claims at High Risk vs. Low Risk
- Losses Prevented through Fraud Alerts
- Agent/Region-wise Fraud Trends

Outcome

- Reduced financial losses from fraudulent claims.
- Faster claim verification with AI support.
- Improved compliance with insurance regulations.
- Enhanced **trust and transparency** between insurers and customers.

PROJECT TITLE : “Insurance Claim Fraud Detection – AI-Powered Monitoring & Alert System”

Phase 1: Problem Understanding & Industry Analysis

Goal: Understand the insurance industry’s pain points in fraud detection and define clear requirements for the system.

Requirement Gathering

Talk to stakeholders (insurance claim officers, policyholders, fraud investigators, compliance teams).

Example Requirements:

- Capture **customer & policy details** in Salesforce.
- Log **insurance claims** with supporting documents.
- Use AI to assign a **fraud risk score** for each claim.
- Automatically create a **fraud investigation case** if risk score exceeds threshold.
- Send alerts to claim officers & compliance teams.
- Generate **fraud detection reports** (losses prevented, high-risk claims, regional patterns).

Stakeholder Analysis

- **Admin (You)** → Manage system setup, workflows, AI model integration.
- **Claim Officers** → Enter and validate claims, review fraud alerts.
- **Fraud Investigators** → Handle flagged claims, resolve fraud cases.
- **Compliance Team** → Ensure adherence to regulatory requirements.
- **Insurance Managers** → Monitor fraud trends, approve high-value claims, analyze dashboards.
- **Policyholders** → Submit claims via portal, upload documents.

Business Process Mapping

Flow of how the system works:

1. Policyholder submits a claim with documents.
2. Claim Officer logs/validates claim in Salesforce.
3. AI model scans data → assigns **Fraud Risk Score**.
    - If **Low Risk** → claim continues normal approval process.
    - If **High Risk** → fraud alert triggered → Case created → assigned to Fraud Investigator.
4. Fraud Investigator reviews evidence & resolves case.
5. Compliance team monitors flagged claims.
6. Manager views fraud prevention metrics via dashboards.

Industry-Specific Use Case Analysis

In the insurance industry:

- Fraud is a **major loss factor** (false claims, staged accidents, fake medical bills).
- Manual detection is **slow, error-prone, and costly**.
- AI can detect patterns (e.g., multiple claims from same customer, mismatched documents).
- Need **real-time alerts** to prevent fraudulent payouts.
- Compliance is crucial → all fraud cases must be auditable.

So, we need to:

- **Automate fraud detection with AI**.
- **Track claims lifecycle & investigations in Salesforce**.
- **Provide managers with dashboards** for insights and risk analysis.

AppExchange Exploration

Look for **Insurance & Fraud Detection apps**:

- "Insurance Policy & Claims Management" (some apps exist but are broad).
- "AI Risk & Fraud Detection" add-ons.
  However, most are **complex enterprise-grade tools**.

We'll build a **simpler custom solution** focused on **fraud detection & case management**, to learn Salesforce + AI integration effectively.

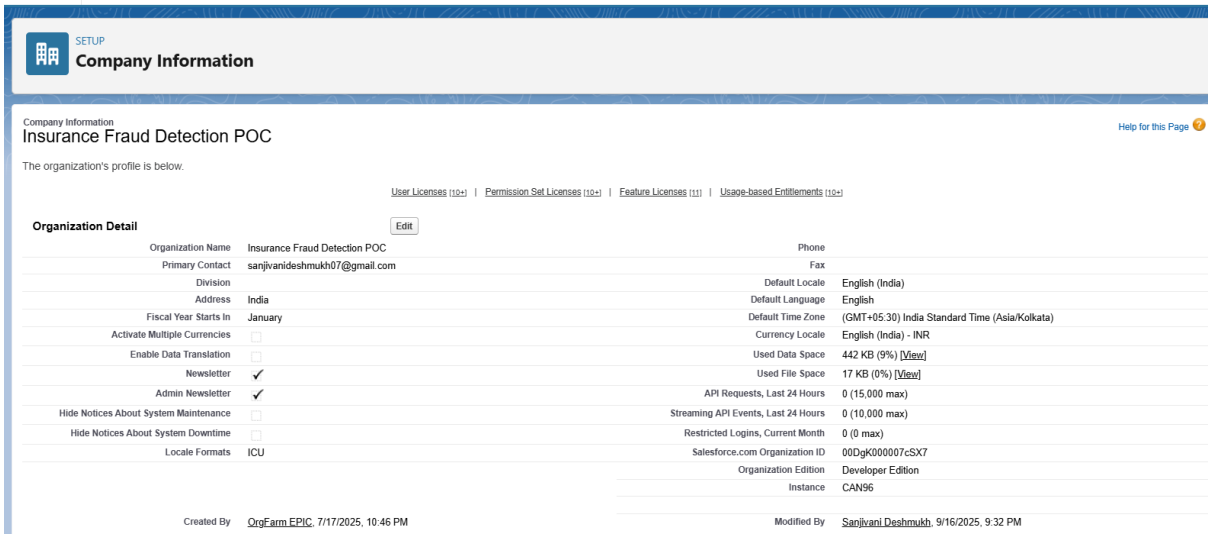
Phase 2: Org Setup & Configuration

1. Salesforce Edition

- Used **Developer Edition Org** for project setup.

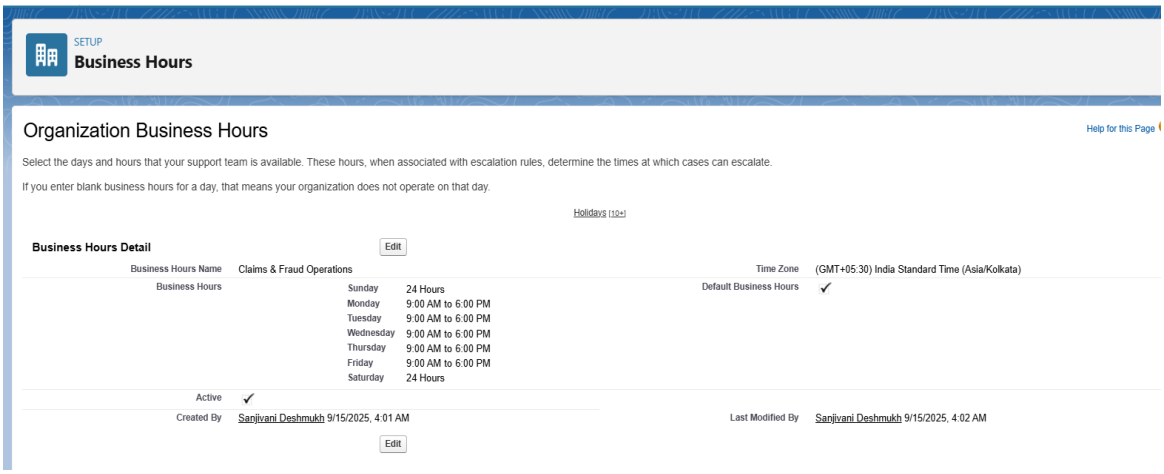
2. Company Profile Setup

- Company Name: *Insurance Fraud Detection POC*
- Primary Contact: Email
- Default Locale: *English (India)*
- Default Time Zone: *Asia/Kolkata (GMT+05:30)*
- Default Currency: *INR*



3. Business Hours & Holidays

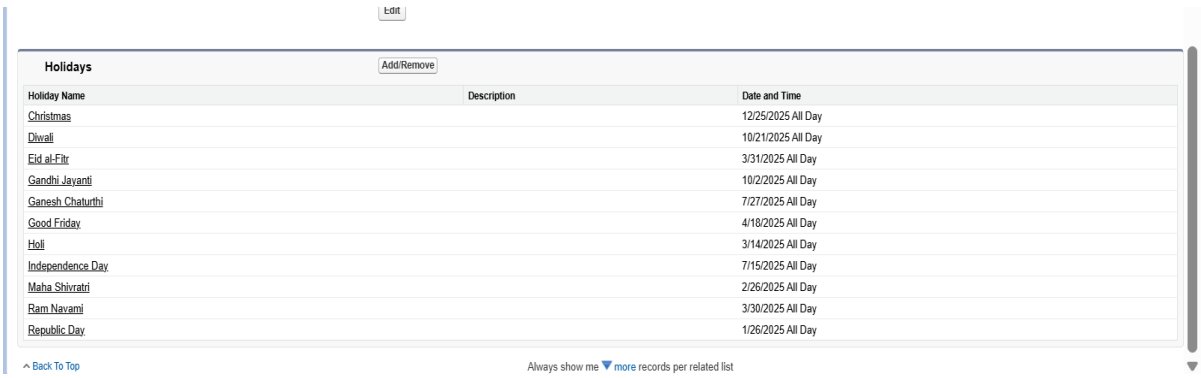
- **Name:** *Claims & Fraud Operations*
- **Time Zone:** *Asia/Kolkata (GMT+05:30)*
- **Default:** ✓
- **Active:** ✓
- **Business Hours:** *Monday – Friday, 9:00 AM – 6:00 PM*
- Leave Sat/Sun unchecked.



Add common public holidays (example for India 2025):

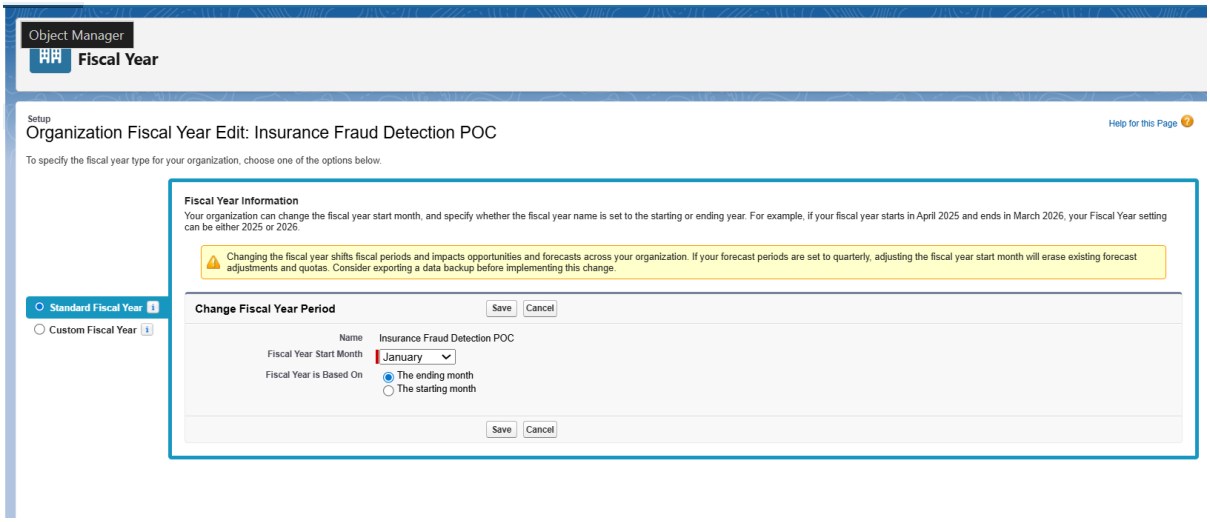
- **Republic Day** – 26 Jan 2025
- **Independence Day** – 15 Aug 2025
- **Gandhi Jayanti** – 2 Oct 2025
- **Diwali** – 21 Oct 2025
- **Christmas** – 25 Dec 2025
- Check **All-Day**.
- Associated each holiday with your *Claims & Fraud Operations* business hours.

Note: This way, Salesforce knows that during these holidays, escalation rules, case clocks, and workflows pause, so fraud investigation SLAs don't count non-working days.



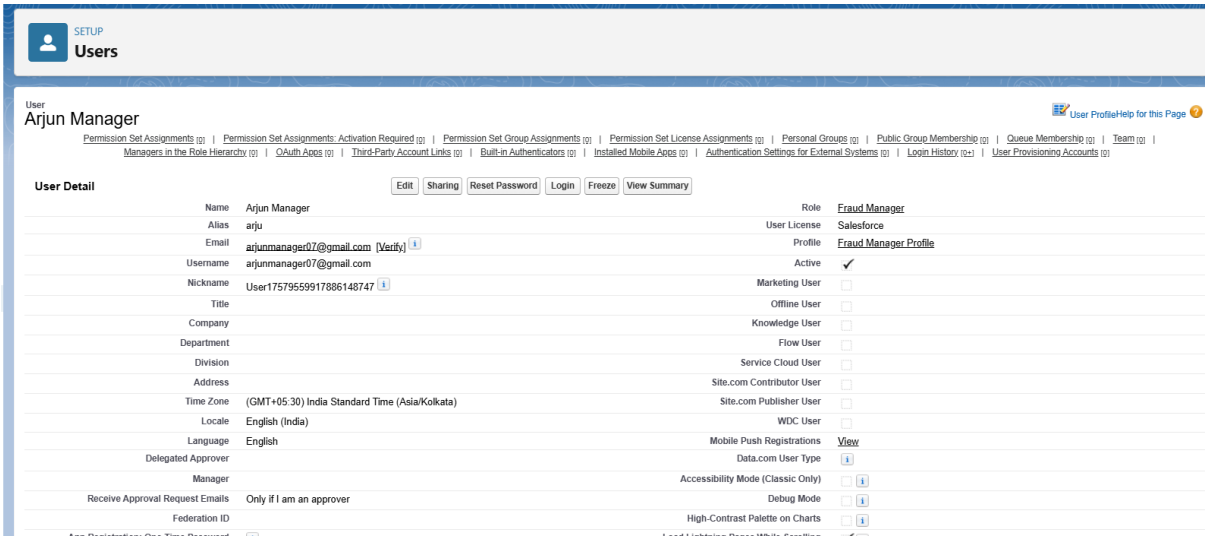
4. Fiscal Year Settings

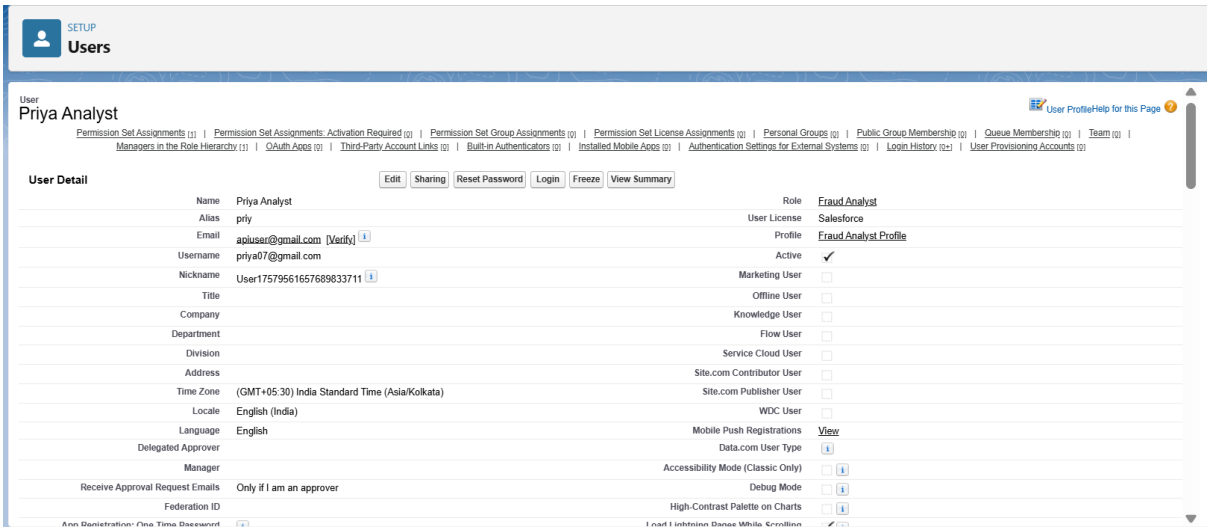
1. Used Standard Fiscal Year (Jan–Dec).

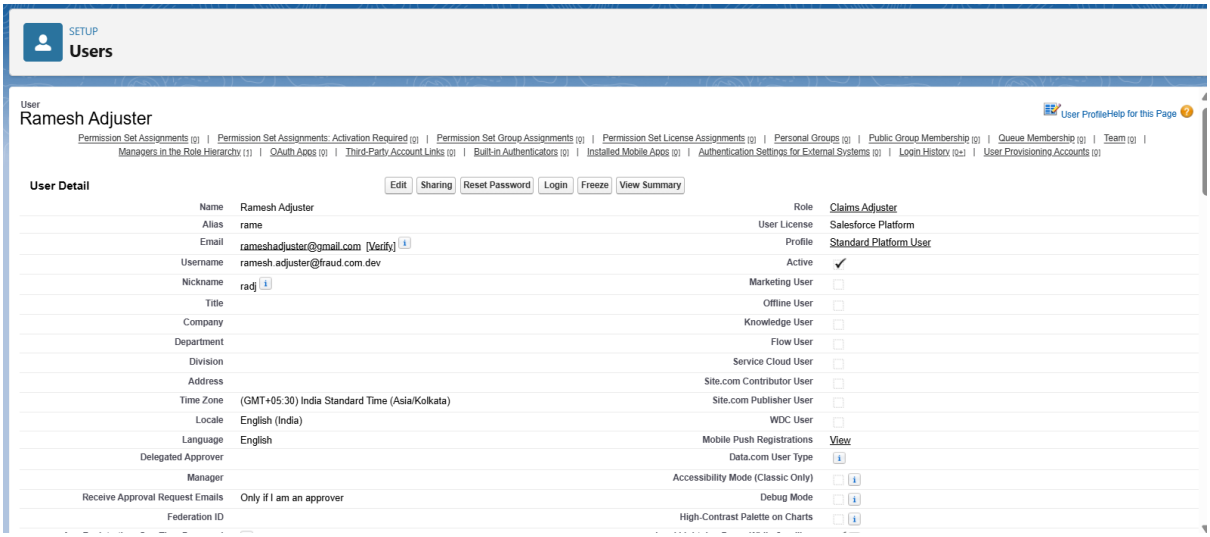


5. User Setup & Licenses

Created Users: Fraud Manager, Fraud Analyst, Claims Adjuster

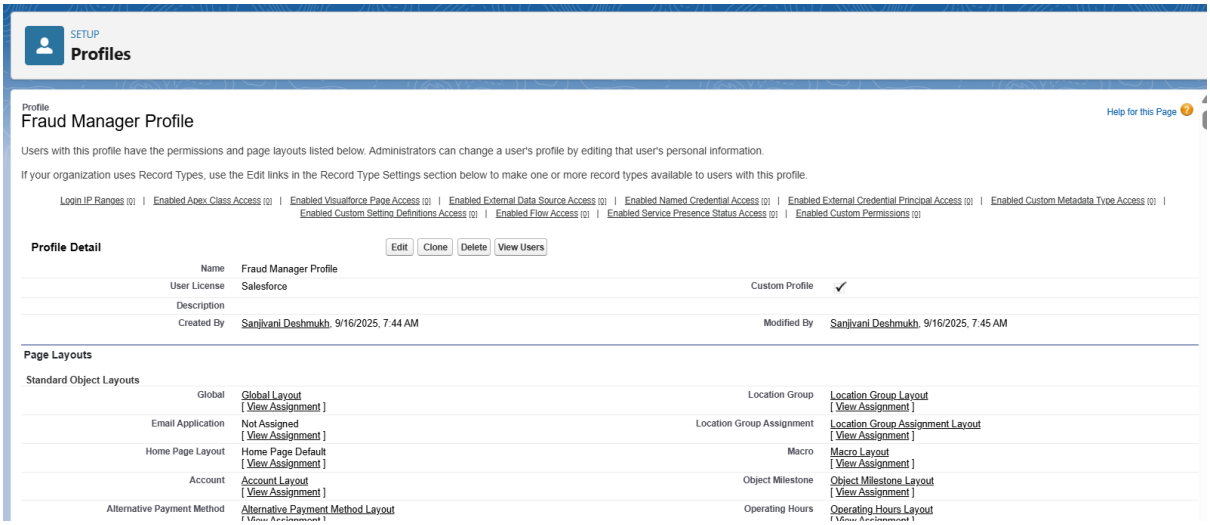




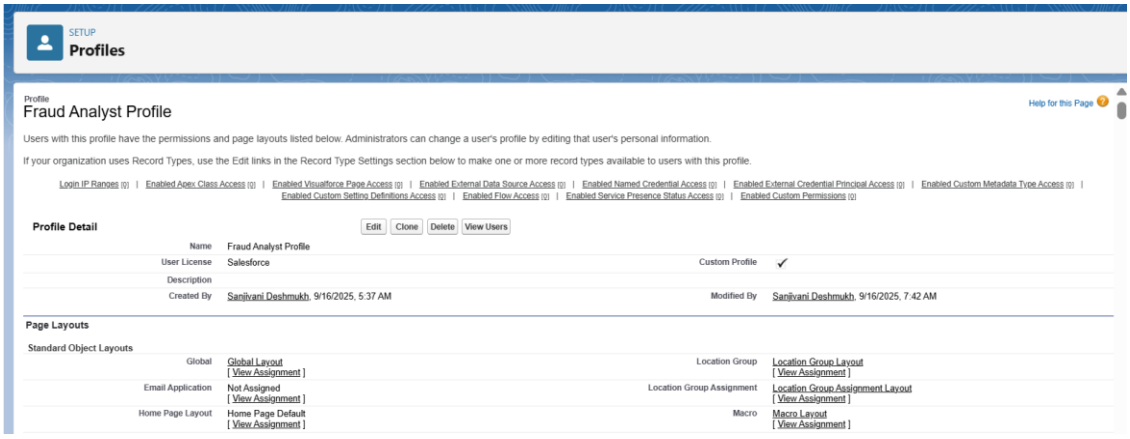


6. Profiles

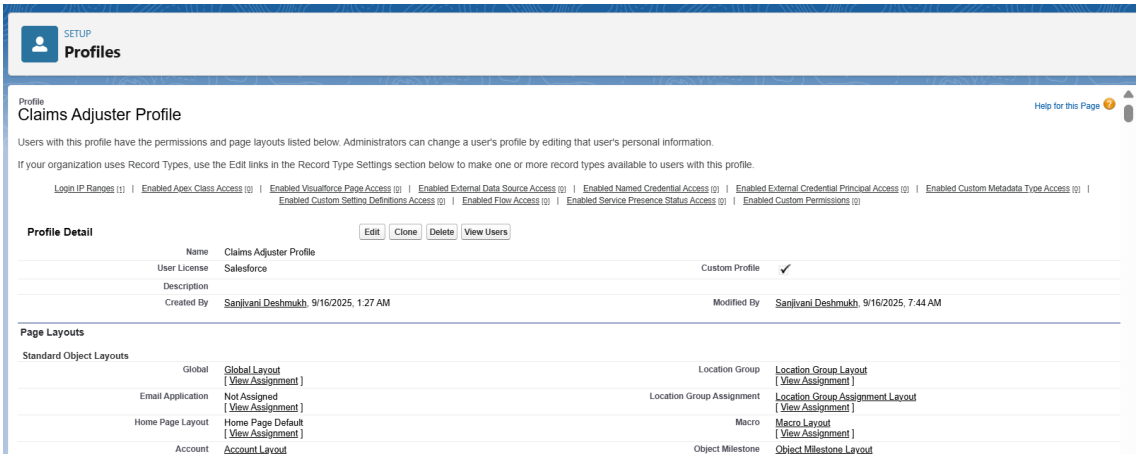
- **Fraud Manager Profile:** Full system access.



- **Fraud Analyst Profile:** Access to fraud analysis objects.

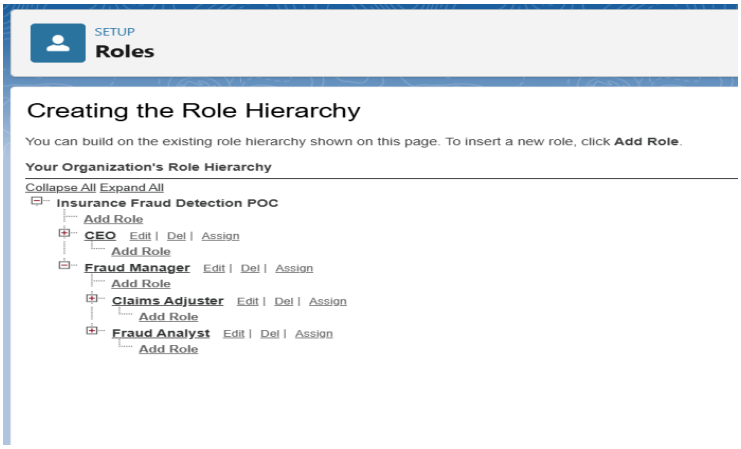


- **Claims Adjuster Profile:** Create & edit claims only.



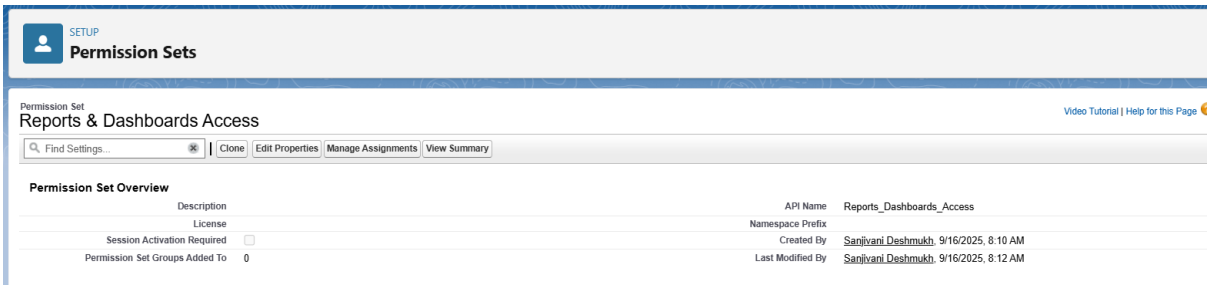
7. Roles

- **Fraud Manager (top)** → full visibility.
- **Fraud Analysts** → below manager.
- **Adjusters** → below manager.



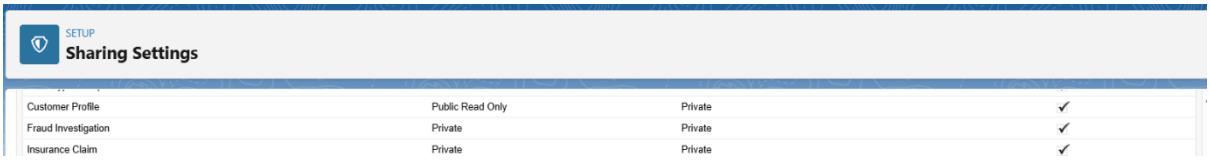
8. Permission Sets

- For Fraud Reporting Access : Extra report/dashboard access.



9. OWD (Org-Wide Defaults)

- **Insurance Claim** → Private (owner & manager only).
- **Customer Profile** → Public Read Only.
- **Fraud Investigation** → Private.

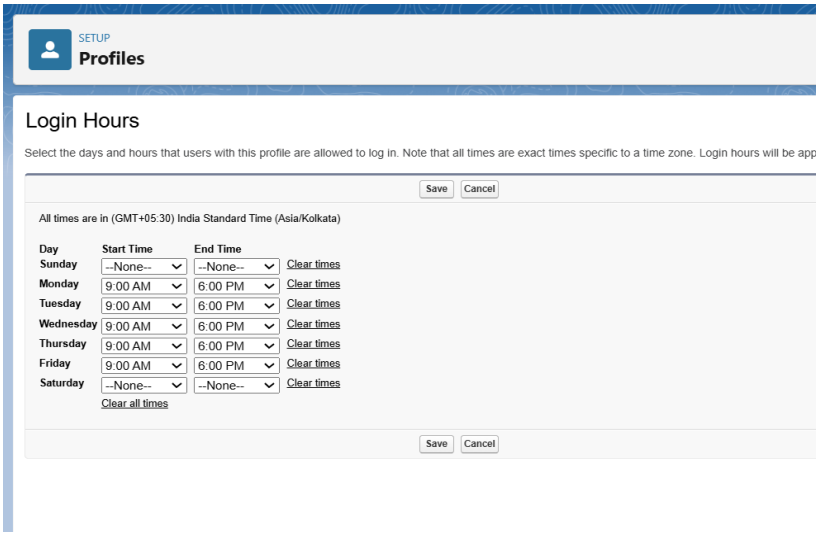


10. Sharing Rules

- Added rules for **Analysts** to view claims created by Adjusters.

11. Login Access Policies

- Restricted working hours login (9 AM – 6 PM) for Adjusters.



12. Dev Org Setup

- Used **Developer Org as Sandbox** for building and testing.

13. Sandbox Usage

- If this were a real company, we'd build in Sandbox, then deploy to Production.

14. Deployment Basics

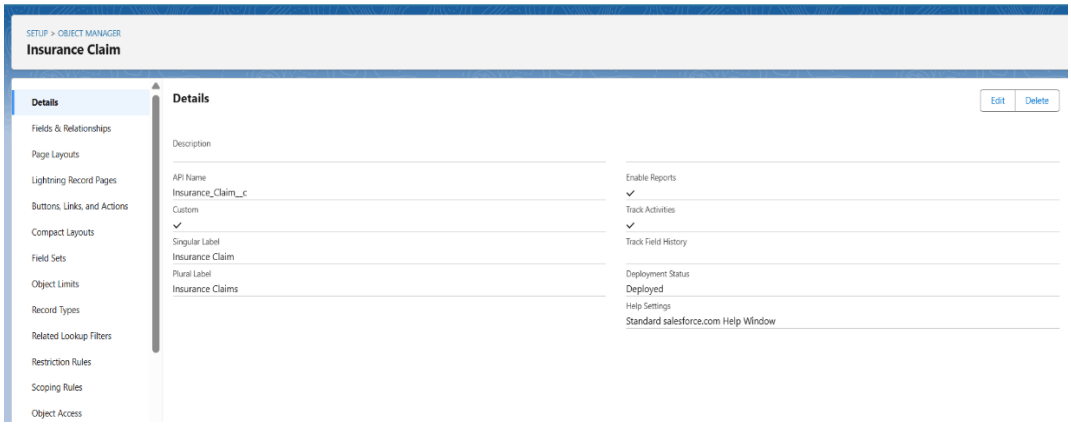
- This project has no Production, as no Sandbox used.

PROJECT TITLE : “Insurance Claim Fraud Detection – AI-Powered Monitoring & Alert System”

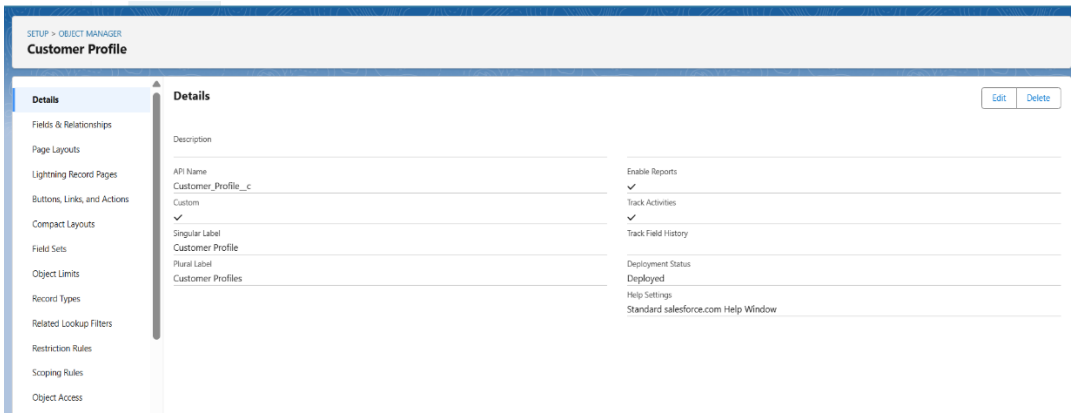
Phase 3: Data Modelling & Relationships

1. Standard & Custom Objects

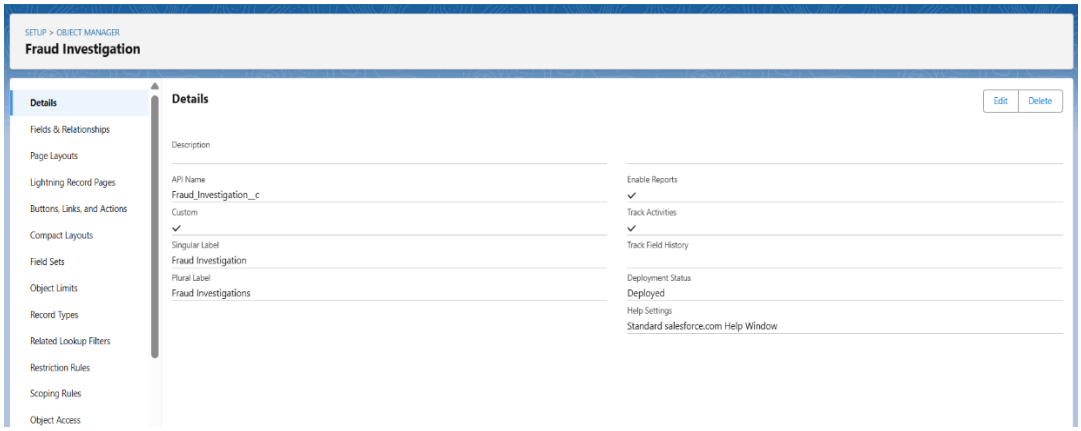
- **Standard Objects we can leverage:**
  - User (Adjuster, Analyst, Manager)
  - Account (could represent Policy Holder or Customer if needed)
- **Custom Objects:**
  - Insurance Claim (tracks claim details)



  - Customer Profile (personal & policy details)



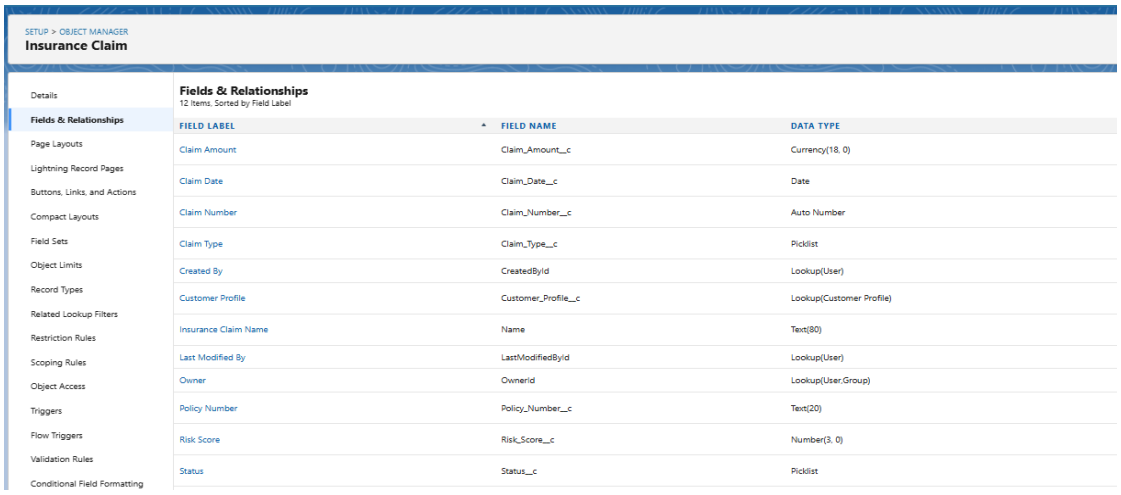
  - Fraud Investigation (fraud case records)



2. Fields:

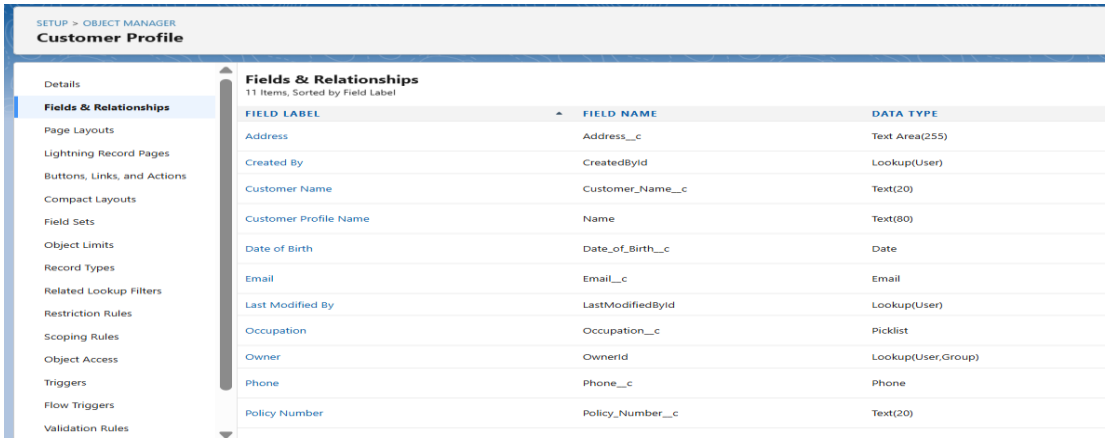
Insurance Claim (Insurance_Claim__c)

- Claim Number → Auto Number (CLM-{0000}) (already set as Record Name)
- Policy Number → Text (20)
- Claim Amount → Currency (16,2)
- Claim Type → Picklist {Accident, Theft, Medical, Other}
- Claim Date → Date
- Status → Picklist {Submitted, In Review, Approved, Rejected, Flagged}
- Risk Score → Number (0–100)



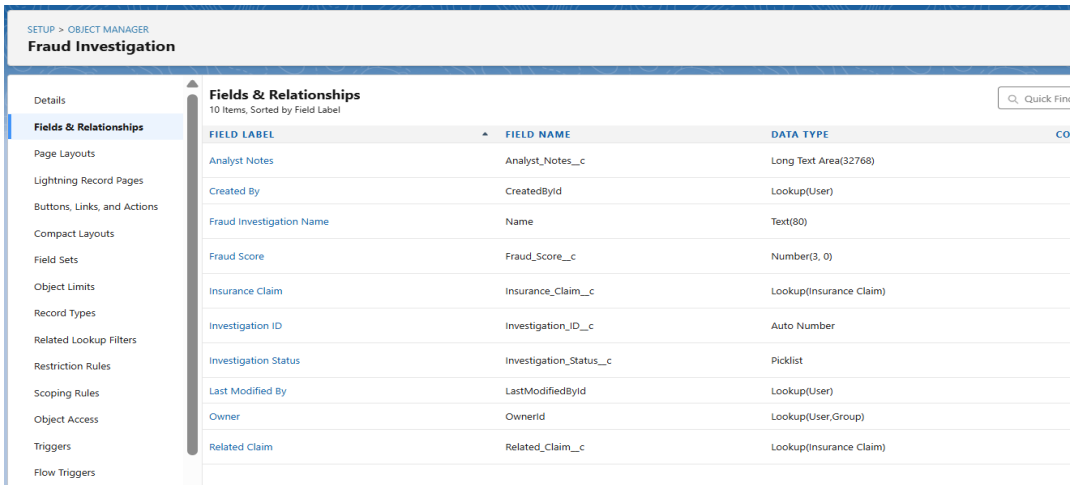
Customer Profile (Customer_Profile__c)

- Customer Name → Text (Record Name)
- Date of Birth → Date
- Email → Email
- Phone → Phone
- Address → Text Area (255)
- Policy Number → Text (20) (must match Claims Policy Number)
- Occupation → Picklist {Salaried, Self-Employed, Business Owner, Retired, Student, Unemployed}



Fraud Investigation (Fraud_Investigation__c)

- Investigation ID → Auto Number (FRA-{0000}) (Record Name)
- Related Claim → Lookup (Insurance Claim)
- Fraud Score → Number (0–100)
- Analyst Notes → Long Text Area (32768, 5 visible lines)
- Investigation Status → Picklist {Open, In Progress, Closed}

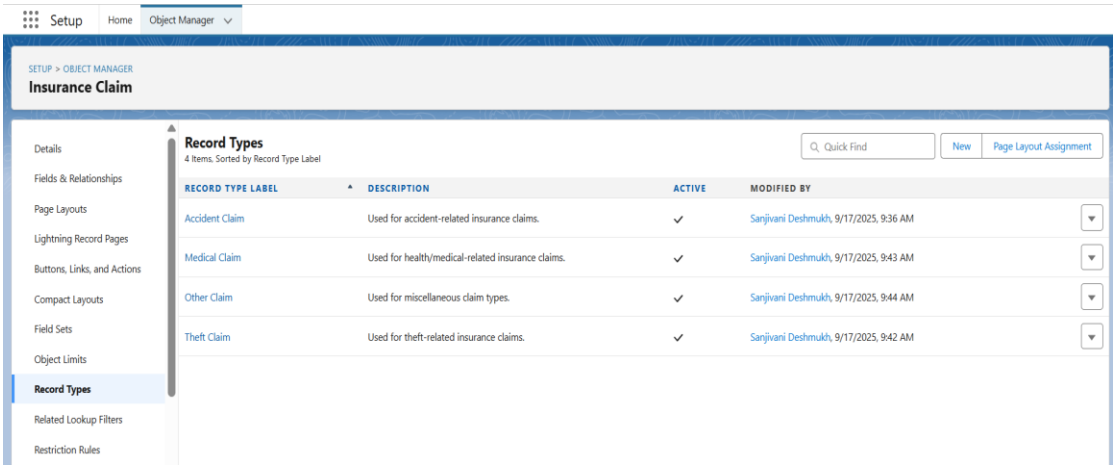


3. Record Types:

Insurance Claim — Record Types

Multiple record types based on claim nature:

- Accident Claim
- Theft Claim
- Medical Claim
- Other Claim



4. Page Layouts:

- **Insurance Claim Layouts**

Accident Claim Layout

- Sections:
  - General Claim Info: Claim Number, Claim Type, Policy Number, Claim Amount, Status
  - Accident Information: Claim Date, Accident Details (custom field), Risk Score
  - System Info: Created By, Last Modified By, Owner

Medical Claim Layout

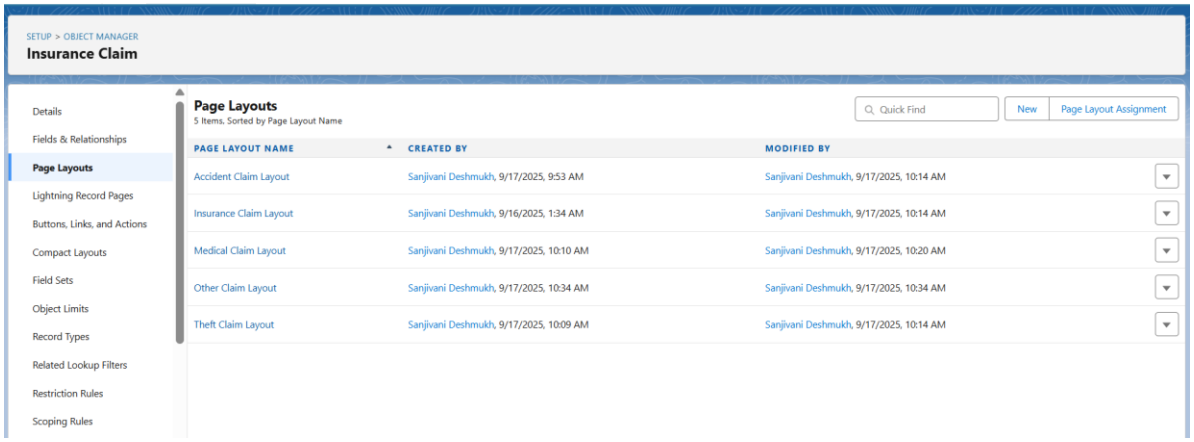
- Sections:
  - General Claim Info: Claim Number, Claim Type, Policy Number, Claim Amount, Status, Claim Date
  - Medical Information: Hospital Name (custom), Treatment Cost (custom), Medical Notes (custom)
  - Risk Assessment: Risk Score
  - System Info: Created By, Last Modified By, Owner

Theft Claim Layout

- Sections:
  - General Claim Info: Claim Number, Claim Type, Policy Number, Claim Amount, Status, Claim Date
  - Theft Information: Police Report Number (custom), Stolen Item Details (custom)
  - Risk Assessment: Risk Score
  - System Info: Created By, Last Modified By, Owner

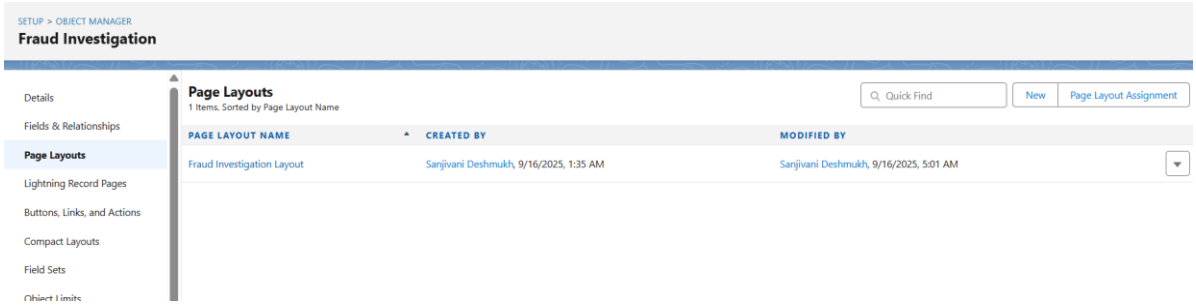
Other Claim Layout

- Sections:
  - General Claim Info: Claim Number, Claim Type, Policy Number, Claim Amount, Status, Claim Date
  - Additional Notes: Generic Notes field
  - Risk Assessment: Risk Score
  - System Info: Created By, Last Modified By, Owner



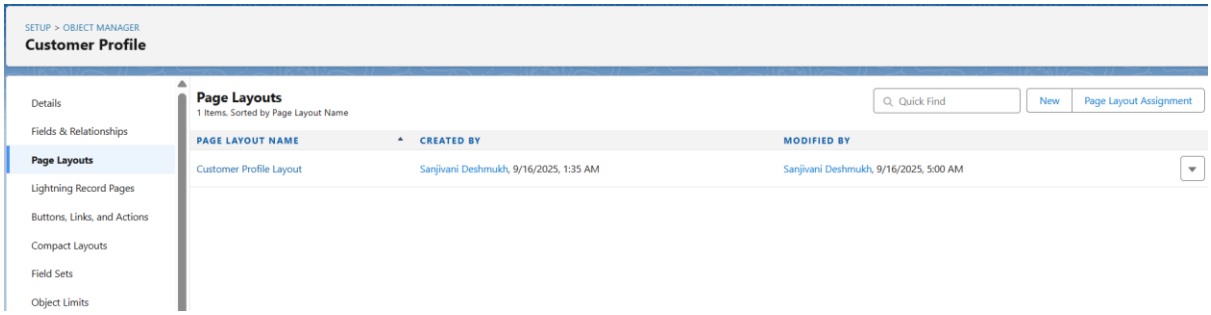
Fraud Investigation Layout (Using Default one)

- General Info: Investigation ID, Related Claim, Fraud Score, Analyst Notes, Investigation Status
- System Info: Created By, Last Modified By, Owner



Customer Profile Layout (Using Default one)

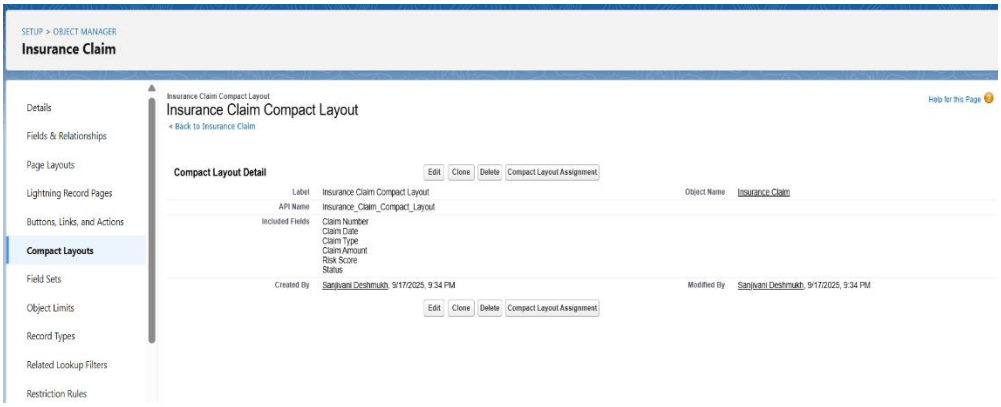
- Customer Details: Customer Name, Date of Birth, Email, Phone, Address, Occupation, Policy Number
- System Info: Created By, Last Modified By, Owner



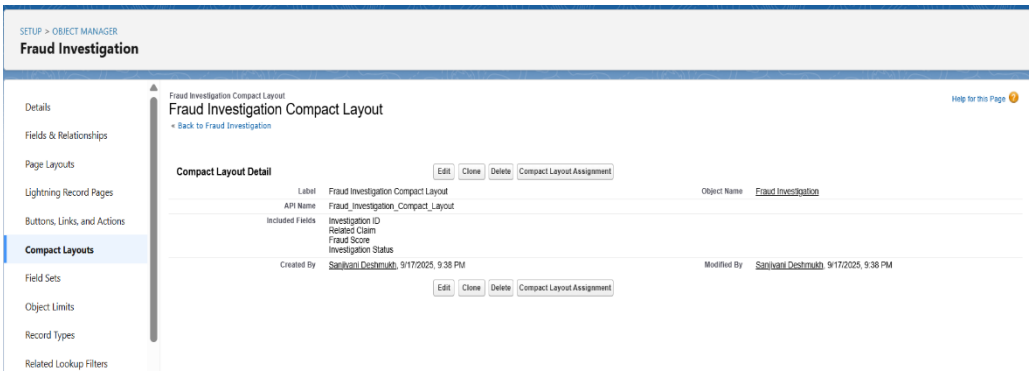
5. Compact Layouts:

This ensures that users see the most critical record information at a glance in the highlight panel.

- **Insurance Claim Compact Layout**
  - Claim Number
  - Claim Type
  - Claim Amount
  - Status
  - Risk Score

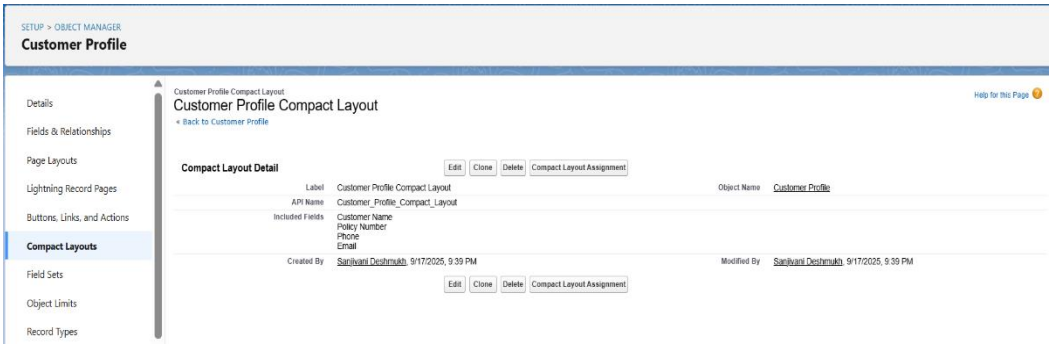


- **Fraud Investigation Compact Layout**
  - Investigation ID
  - Related Claim
  - Fraud Score
  - Investigation Status



- **Customer Profile Compact Layout**

- Customer Name
- Policy Number
- Phone
- Email



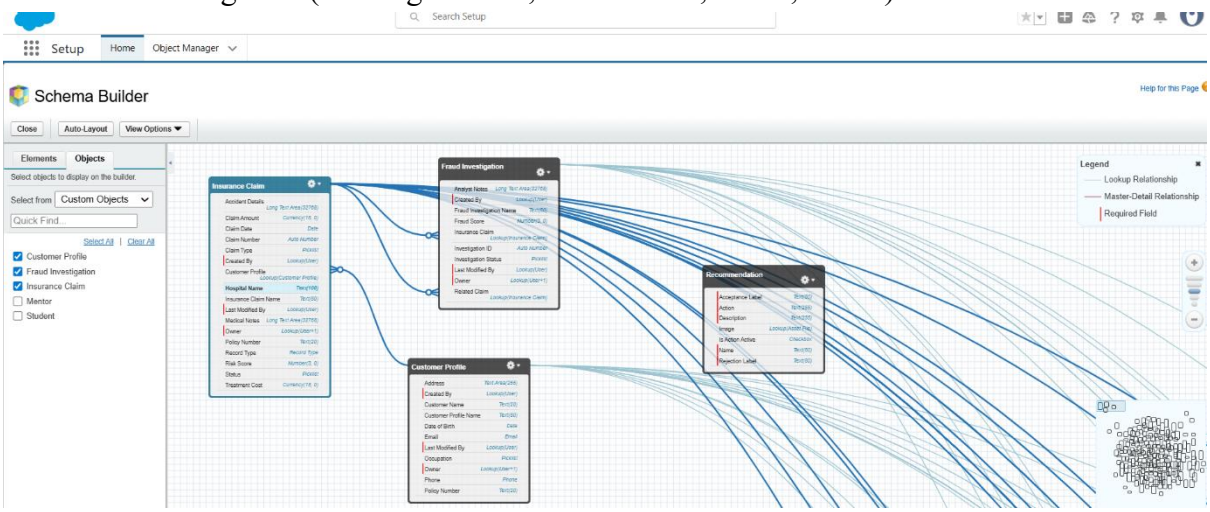
6. Schema Builder

Schema View

- Customer Profile (Policy Number, Name, DOB, Email, Phone, etc.)

  ⬇️ Lookup →
- Insurance Claim (Claim Number, Claim Type, Claim Amount, Claim Date, Status, Risk Score, Accident/Medical/Theft Details)

  ⬇️ Lookup →
- Fraud Investigation (Investigation ID, Fraud Score, Notes, Status)



7. Junction Objects:

No junction objects created, junction objects could be added for advanced scenarios like if we want to track **multiple Customers linked to a single Claim** or if we want to link a **single Investigation to multiple Claim.**

8. External Objects:

No external Objects created, could be added for real-time fraud scoring by connecting Salesforce with external transaction data sources.

TITLE: “Insurance Claim Fraud Detection – AI-Powered Monitoring & Alert System”

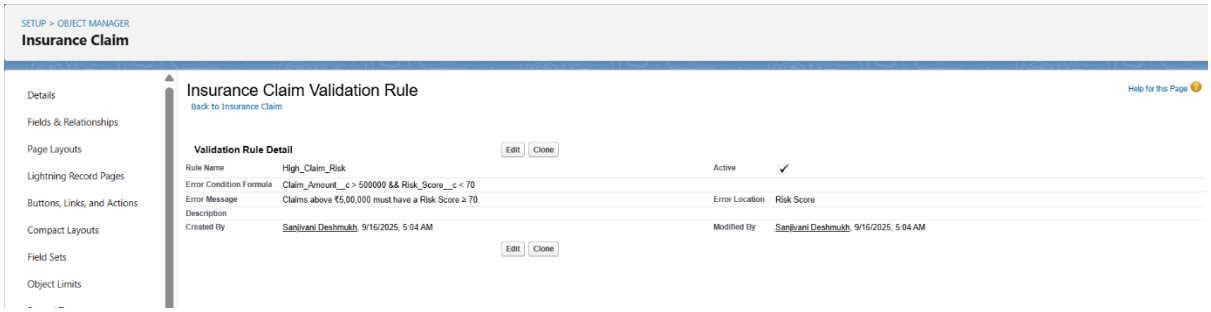
Phase 4: Process Automation (Admin)

Goal: Automate claim-handling, fraud detection and escalation so suspicious claims are flagged, routed and tracked with minimal manual effort.

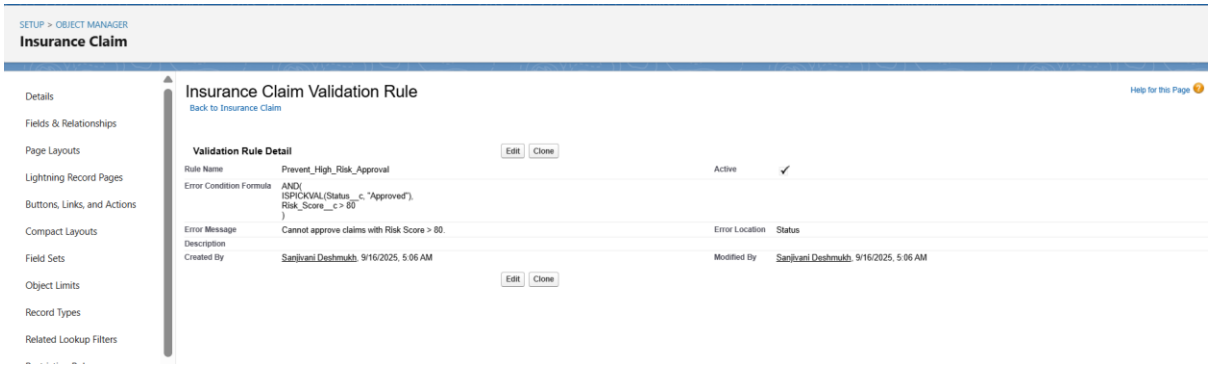
1. Validation Rules:

- **Purpose:** Enforce data quality and prevent risky actions (e.g., approving a high-risk claim). These run at save-time and block invalid states.

- **High_Claim_Risk_Score** — prevent saving large claims with low risk assessment.
    - **Logic:** if Claim_Amount__c > 500000 AND Risk_Score__c < 80 then block.
    - Formula: AND(
      Claim_Amount__c > 500000,
      Risk_Score__c < 80
)
    - **Error message:** “High claim amount requires Risk Score ≥ 80.” (placed on Risk Score)



- **Approved_High_Risk_Not_Allowed** — prevent approving a claim with very high risk.
    - **Logic:** if Status = Approved AND Risk Score > 80 then block.
    - Formula: AND(
      ISPICKVAL(Status__c, "Approved"),
      Risk_Score__c > 80
)
    - **Error message:** “Claims with Risk Score > 80 cannot be approved.” (placed on Status)



2. Workflow Rules (legacy):

- **Purpose:** Workflow Rules in Salesforce help automate immediate actions when certain conditions are met (like sending alerts, updating fields, or assigning tasks).
- **Example of Workflow Rule**

High-Risk Claim Notification (Implemented in Flows)

- **Object:** Insurance Claim
- **Rule Criteria:** Risk Score ≥ 70
- **Action:**
  - Send **Email Alert** to the Fraud Analyst.
  - Create a **Task** for the Adjuster: *"Review High-Risk Claim."*

3. Process Builder (legacy):

- **Purpose:** Process Builder allows multiple conditions, multiple actions, and record updates across objects.
- **Example of Process Builder**

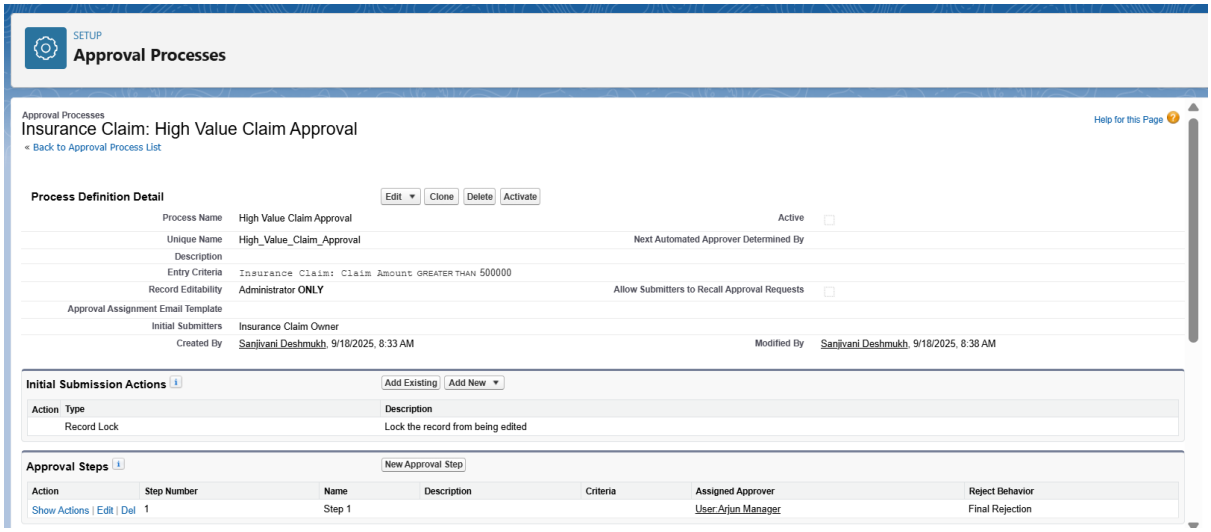
Auto-Update Claim Status on High Fraud Score (Implemented in Flows)

- **Trigger:** When Fraud Investigation is created/updated.
- **Condition:** Fraud Score ≥ 70.
- **Action:** Update related Insurance Claim → Status = *"In Review"*.

4. Approval Process:

- **Purpose:** Require manager approval for high-value claims (compliance & control). Locks the record until final decision.
- **Approval process name: High Value Claim Approval**
  - **Entry criteria:** Claim_Amount__c > 500000

- **Approver:** Fraud Manager (can be a specific user or manager role)
- **Initial submission action:** lock record, send submission email to submitter
- **Final approval actions:**
- **Field Update — Claim Status = Approved** (Field Update name: Set_Status_Approved)
- **Email Alert — Approval Confirmation to Customer** (template: *Approval Confirmation*)
- **Final rejection actions:**
- **Field Update — Claim Status = Rejected** (Field Update name: Set_Status_Rejected)
- **Email Alert — Notify Analyst of Rejection**

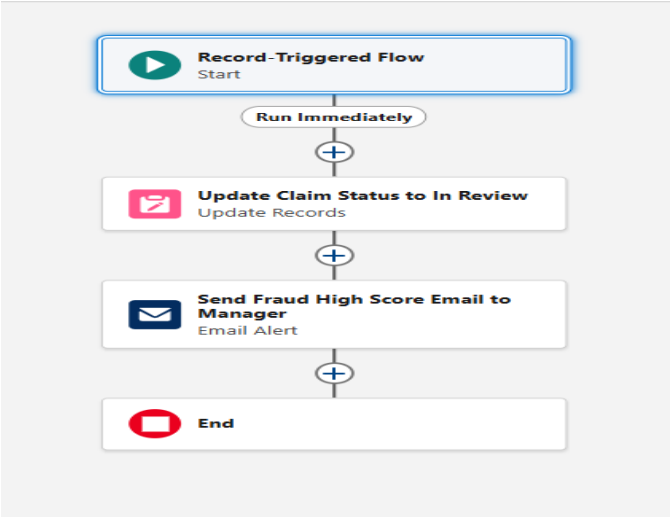


5. Flow Builder:

- **Purpose:** All active automation is implemented in Flow Builder. Created Record-Triggered Flows and a Scheduled-Triggered Flow.

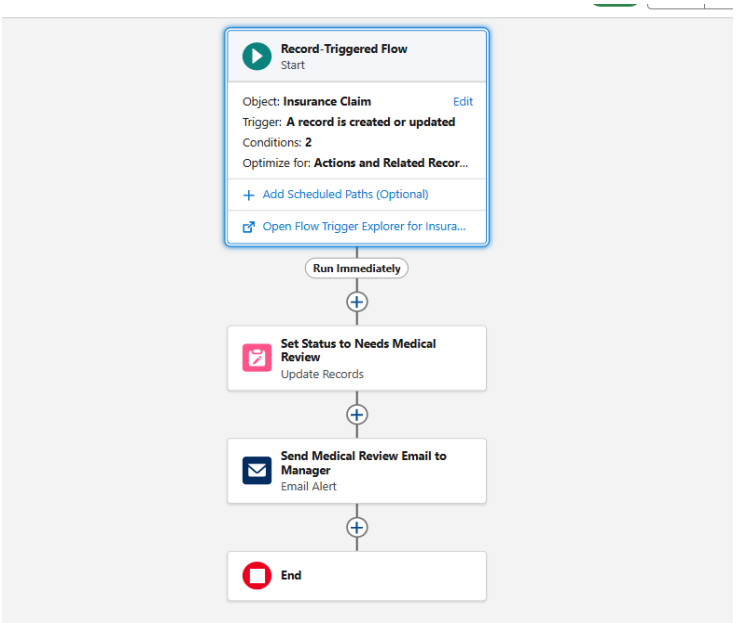
Flow A — Fraud_Score_AutoFlag_Flow (Record-Triggered Flow)

- **Object:** Fraud_Investigation
- **Trigger:** When record is created or updated
- **Entry condition:** `Fraud_Score__c >= 70`
- **Optimize:** Actions and Related Records
- **Immediate actions:**
  - **Update Records** → find **Invoice Claim** where `Id = $Record.Related_Claim__c` → set `Status__c = "In Review"`.
  - **Action (Email Alert)** → send `Fraud High Score Alert to Manager`.



Flow B — Medical_Claim_Review_Flow (Record-Triggered Flow)

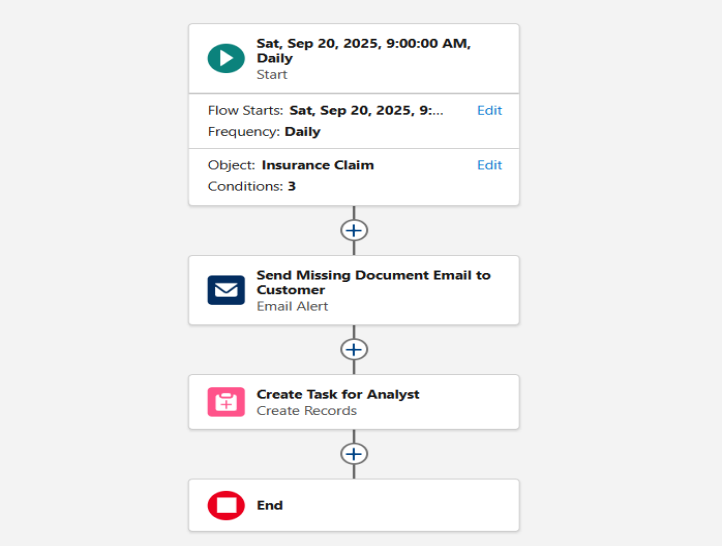
- **Object:** Insurance_Claim
- **Trigger:** When record is created or updated
- **Entry condition:** ISPICKVAL(Claim_Type__c,"Medical") AND Claim_Amount__c > 200000
- **Immediate actions:**
    1. **Update Records** → use the record that triggered the flow → set Status__c = "Needs Medical Review".
    2. **Action (Email Alert)** → send Medical Review Alert to Manager.



Flow C — Missing_Documents_Reminder_Flow (Scheduled-Triggered Flow)

- **Schedule:** Daily at 9:00 AM
- **Object:** Insurance_Claim

- **Condition:** ISPICKVAL(Status__c,"Submitted") AND ISPICKVAL(Document_Status__c,"Pending") AND CreatedDate <= (TODAY() - 3)
- **Actions per matching record:**
    1. **Action (Email Alert)** → Missing Document Reminder (recipient: Customer email from related Customer Profile).
    2. **Create Records** → Task: Subject = "Follow up on missing claim documents", OwnerId = Analyst user or Analyst Queue, ActivityDate = TODAY() + 1, WhatId = claim Id.

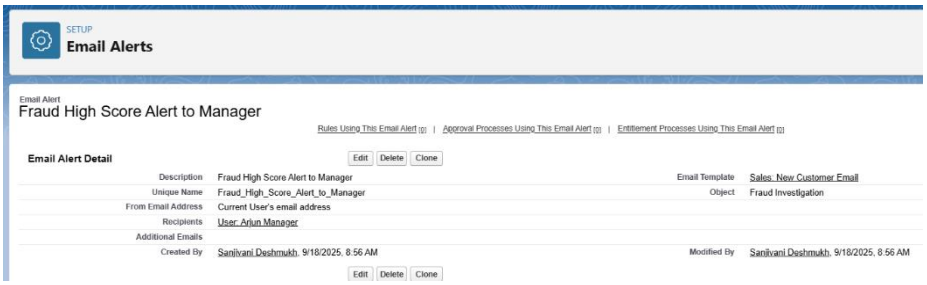


6. Email Alerts

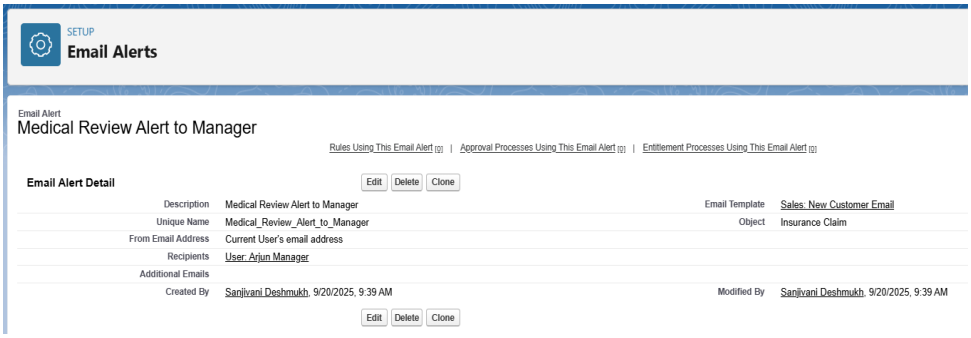
- **Purpose:** Reusable email-alert records (with templates) are easiest to call from Flows/Approval processes.

Email alerts created (examples and recipients)

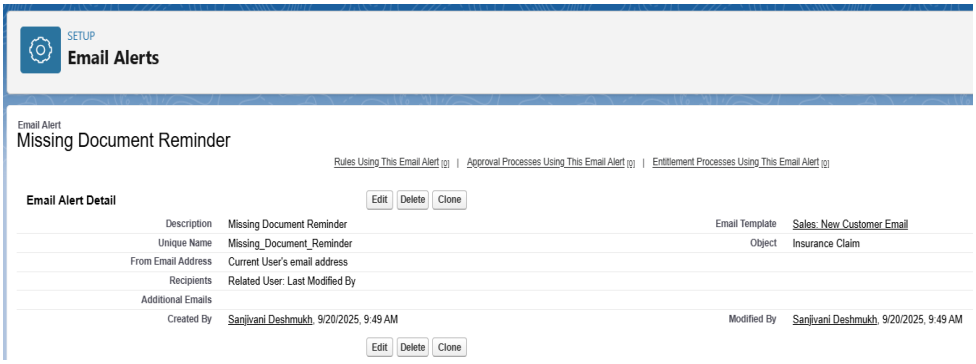
- **Fraud High Score Alert to Manager** → object: Fraud Investigation → template: *Fraud High Score Alert* → recipients: Fraud Manager (User or Role).



- **Medical Review Alert to Manager** → object: Insurance Claim → template: *Medical Claim Needs Review* → recipient: Manager.



- **Missing Document Reminder** → object: Insurance Claim → template: *Please upload missing documents* → recipient: related Customer email (Customer_Profile__r.Email__c) or direct Contact lookup.

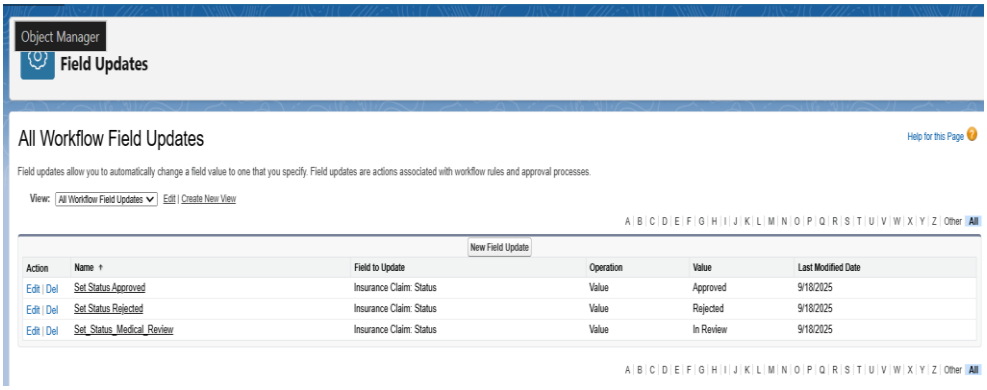


7.Field Updates:

Purpose: Automatic field updates are performed inside Flows or as Approval Process field update actions.

Examples implemented

- **In Flows:** Update Status__c to **In Review** or **Needs Medical Review** (Flow elements).
- **In Approval Process:** Final Approval action sets Status__c = Approved; Final Rejection sets Status__c = Rejected.

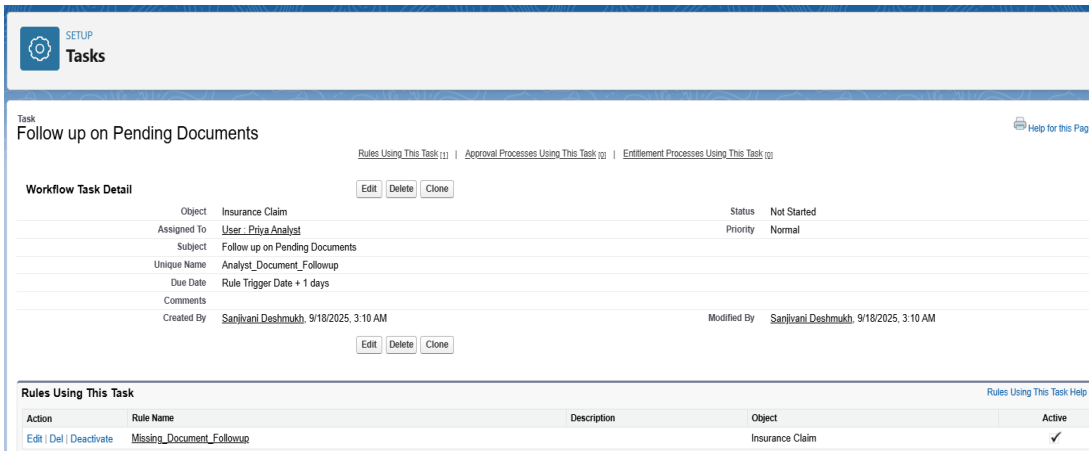


8. Tasks:

Purpose: Create follow-up work items for analysts and adjusters when human action is needed.

Examples implemented

- **Scheduled Flow** → create Task for Analyst: Subject "Follow up on missing claim documents", Related To = claim, Owner = Analyst user or Queue, ActivityDate = TODAY() + 1.



TITLE: “Insurance Claim Fraud Detection – AI-Powered Monitoring & Alert System”

Phase 5: Apex Programming (Developer)

Goal: Implement server-side logic to support complex/frequent operations that cannot be handled (or are inefficient) with clicks alone — bulk-safe, test-covered, and maintainable code for fraud detection and claim processing.

1. Classes & Objects:

Purpose

Contain business logic in reusable, testable units (Apex classes) and keep triggers thin.

Implementation

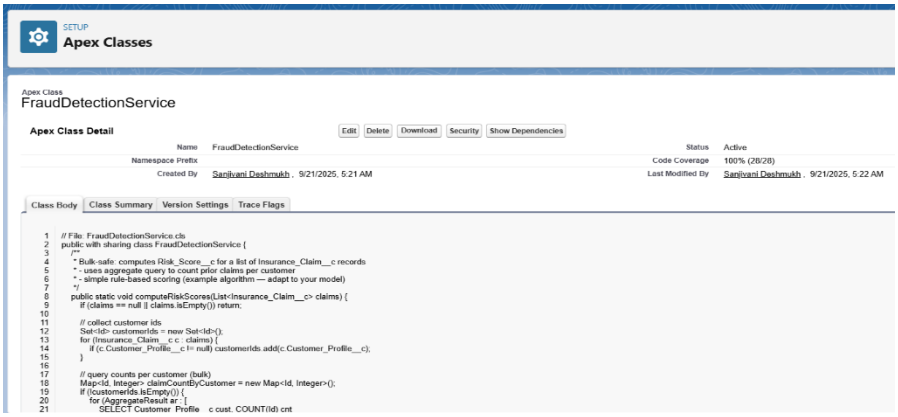
- **FraudDetectionService** — core business logic to calculate **Risk_Score__c** for **Insurance_Claim__c** records in bulk.
- Small utility classes where needed (e.g., email helpers, constants).

Why

- **Encapsulation:** single place to change scoring logic.
- **Reusability:** service can be called from triggers, batches, schedulers, tests.
- **Testability:** easy to unit test pure logic.

Implementation steps

1. Create a new Apex Class in Developer Console (File → New → Apex Class) named **FraudDetectionService.**
2. Implement a bulk-safe public static method like **computeRiskScores**(**List<Insurance_Claim__c>** claims) that updates **Risk_Score__c** in-memory.
3. Keep no DML inside loops; use aggregate queries to fetch supporting data.



2. Apex Triggers (before/after insert/update/delete)

Purpose

React to DML events to run server logic before or after records are saved.

Implementation

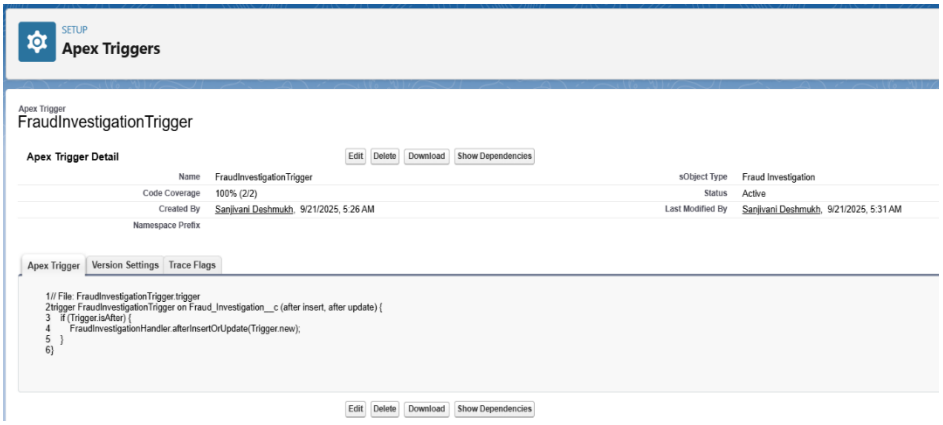
- **InsuranceClaimTrigger** (before insert, before update, after insert, after update) — delegates to handler.
    - Before: call **FraudDetectionService.computeRiskScores()** to set **Risk_Score__c**.
    - After insert/update: create **Fraud_Investigation__c** if claim is flagged/high-risk.
- **FraudInvestigationTrigger** (after insert, after update) — delegates to handler to update related claim(s) status and enqueue notifications.

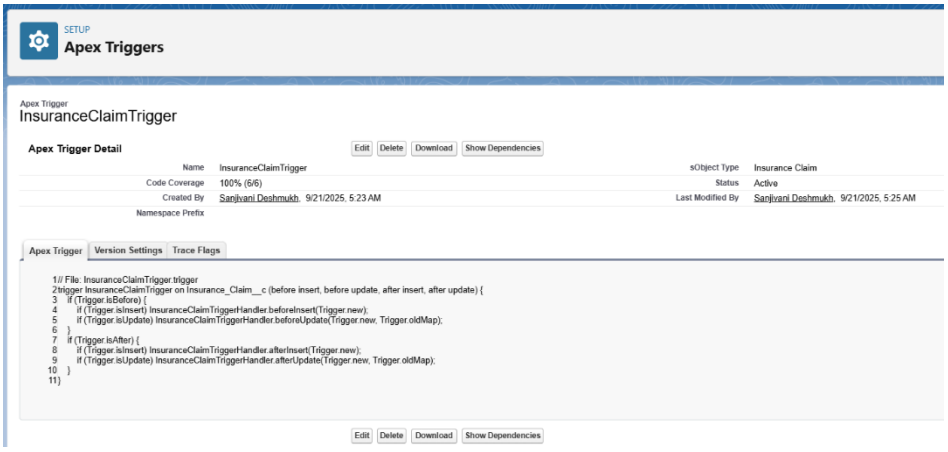
Why

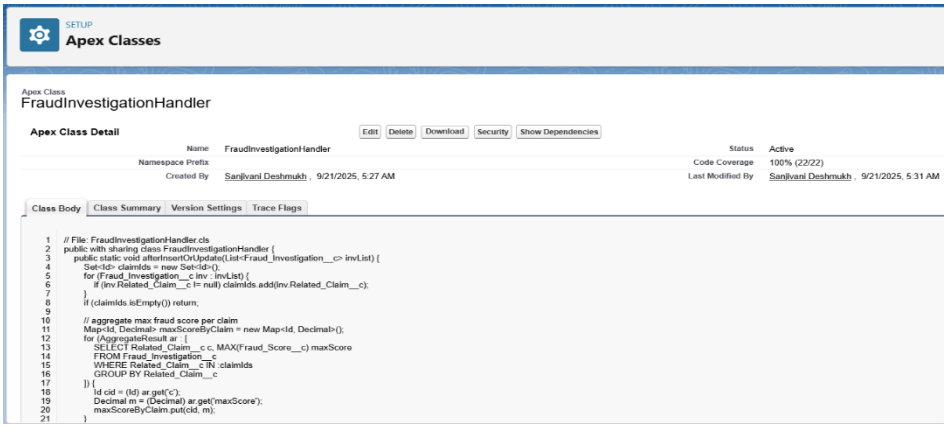
- before triggers update fields on the same record prior to DML (efficient).
- after triggers create related records (need Ids) and perform cross-object updates.

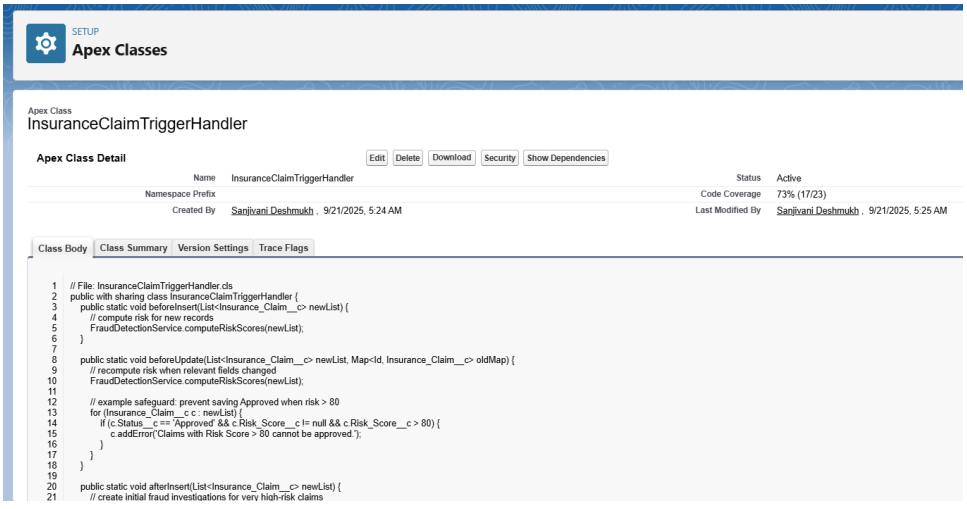
Implementation steps

1. Create trigger file (Developer Console → New → Apex Trigger). only delegate to a handler.
2. Create a handler class **InsuranceClaimTriggerHandler** and **FraudInvestigationHandler** with public static methods for each event.
3. Ensure bulk-safe coding: pass List<...> into handler methods and process collections.


SETUP
Apex Triggers
Apex Trigger
FraudInvestigationTrigger
Apex Trigger Detail
Edit Delete Download Show Dependencies
Name FraudInvestigationTrigger
sObject Type Fraud Investigation
Code Coverage 100% (2/2)
Status Active
Created By Sanjivani Deshmukh, 9/21/2025, 5:26 AM
Last Modified By Sanjivani Deshmukh, 9/21/2025, 5:31 AM
Namespace Prefix
Apex Trigger Version Settings Trace Flags
1// File: FraudInvestigationTrigger.trigger
2trigger FraudInvestigationTrigger on Fraud_Investigation__c (after insert, after update) {
3 if (Trigger.isAfter) {
4 FraudInvestigationHandler.afterInsertOrUpdate(Trigger.new);
5 }
6}
Edit Delete Download Show Dependencies


SETUP
Apex Triggers
Apex Trigger
InsuranceClaimTrigger
Apex Trigger Detail
Edit Delete Download Show Dependencies
Name InsuranceClaimTrigger
sObject Type Insurance Claim
Code Coverage 100% (6/6)
Status Active
Created By Sanjivani Deshmukh, 9/21/2025, 5:23 AM
Last Modified By Sanjivani Deshmukh, 9/21/2025, 5:25 AM
Namespace Prefix
Apex Trigger Version Settings Trace Flags
1// File: InsuranceClaimTrigger.trigger
2trigger InsuranceClaimTrigger on Insurance_Claim__c (before insert, before update, after insert, after update) {
3 if (Trigger.isBefore) {
4 if (Trigger.isInsert) InsuranceClaimTriggerHandler.beforeInsert(Trigger.new);
5 if (Trigger.isUpdate) InsuranceClaimTriggerHandler.beforeUpdate(Trigger.new, Trigger.oldMap);
6 }
7 if (Trigger.isAfter) {
8 if (Trigger.isInsert) InsuranceClaimTriggerHandler.afterInsert(Trigger.new);
9 if (Trigger.isUpdate) InsuranceClaimTriggerHandler.afterUpdate(Trigger.new, Trigger.oldMap);
10 }
11}
Edit Delete Download Show Dependencies


SETUP
Apex Classes
Apex Class
FraudInvestigationHandler
Apex Class Detail
Edit Delete Download Security Show Dependencies
Name FraudInvestigationHandler
Status Active
Namespace Prefix
Code Coverage 100% (22/22)
Created By Sanjivani Deshmukh, 9/21/2025, 5:27 AM
Last Modified By Sanjivani Deshmukh, 9/21/2025, 5:31 AM
Class Body Class Summary Version Settings Trace Flags
1 // File: FraudInvestigationHandler.cls
2 public with sharing class FraudInvestigationHandler {
3 public static void afterInsertOrUpdate(List<Fraud_Investigation__c> invList) {
4 Set<Id> claimIds = new Set<Id>();
5 for (Fraud_Investigation__c inv : invList) {
6 if (inv.Related_Claim__c != null) claimIds.add(inv.Related_Claim__c);
7 }
8 if (claimIds.isEmpty()) return;
9
10 // aggregate max fraud score per claim
11 Map<Id, Decimal> maxScoreByClaim = new Map<Id, Decimal>();
12 for (AggregateResult ar : [
13 SELECT Related_Claim__c c, MAX(Fraud_Score__c) maxScore
14 FROM Fraud_Investigation__c
15 WHERE Related_Claim__c IN :claimIds
16 GROUP BY Related_Claim__c
17]) {
18 Id cid = (Id) ar.get('c');
19 Decimal m = (Decimal) ar.get('maxScore');
20 maxScoreByClaim.put(cid, m);
21 }


SETUP
Apex Classes
Apex Class
InsuranceClaimTriggerHandler
Apex Class Detail
Edit Delete Download Security Show Dependencies
Name InsuranceClaimTriggerHandler
Status Active
Namespace Prefix
Code Coverage 73% (17/23)
Created By Sanjivani Deshmukh, 9/21/2025, 5:24 AM
Last Modified By Sanjivani Deshmukh, 9/21/2025, 5:25 AM
Class Body Class Summary Version Settings Trace Flags
1 // File: InsuranceClaimTriggerHandler.cls
2 public with sharing class InsuranceClaimTriggerHandler {
3 public static void beforeInsert(List<Insurance_Claim__c> newList) {
4 // compute risk for new records
5 FraudDetectionService.computeRiskScores(newList);
6 }
7
8 public static void beforeUpdate(List<Insurance_Claim__c> newList, Map<Id, Insurance_Claim__c> oldMap) {
9 // recompute risk when relevant fields changed
10 FraudDetectionService.computeRiskScores(newList);
11
12 // example safeguard: prevent saving Approved when risk > 80
13 for (Insurance_Claim__c c : newList) {
14 if (c.Status__c == 'Approved' && c.Risk_Score__c != null && c.Risk_Score__c > 80) {
15 c.addError('Claims with Risk Score > 80 cannot be approved.');
16 }
17 }
18 }
19
20 public static void afterInsert(List<Insurance_Claim__c> newList) {
21 // create initial fraud investigations for very high-risk claims

3. Trigger Design Pattern:

Purpose

Organize trigger logic to be maintainable, testable and avoid duplication.

Implementation

- Thin trigger files delegating to a ***Handler** class. Handler methods are organized by context (**beforeInsert, beforeUpdate, afterInsert, afterUpdate**).

Why

- Keeps triggers small and readable.
- Easier to unit test handler methods.
- Supports reuse and avoids mixed responsibilities.

Implementation notes

- Handler methods accept collections (e.g., **List<Insurance_Claim__c> newList**), and when needed **Map<Id, SObject> oldMap**.
- Add a central Handler class that calls the **FraudDetectionService** or other service classes.

4. SOQL & SOSL:

Purpose

Query data from Salesforce; SOSL for text search across objects.

Implementation

- SOQL aggregated query to count prior claims per customer (used in scoring).
- SOQL to fetch Users (e.g., managers) when sending notifications.
- No heavy SOSL needed in current scope; mention available for future search use-cases.

Why

- Required to compute counts and to find related records (**Fraud_Investigation__c** aggregates or find managers).

Implementation steps

- Use aggregate queries for counts:

- List<AggregateResult> arList = [SELECT Customer_Profile__c c, COUNT(Id) cnt

  FROM Insurance_Claim__c WHERE Customer_Profile__c IN :customerIds GROUP BY Customer_Profile__c];

    - Always limit fields and records returned; use **WHERE** clauses to reduce data.

5. Collections: List, Set, Map:

Purpose

Efficiently store and manipulate data in bulk operations.

Used

- **Set<Id>** to collect unique **Customer_Profile__c ids**.
- **Map<Id, Integer>** to store counts per customer.
- **List<Insurance_Claim__c>** for DML operations.

Why

- Prevent duplicates (Set), fast lookup (Map), ordered DML (List).

Implementation notes

- Build sets from trigger records, then query using IN :set.
- Map aggregate results to a map for O(1) lookup.

6. Control Statements:

Purpose

Implement decision logic (if/else, for, while, switch).

Used

- if-else to apply score buckets and to decide when to create investigations.
- for loops to iterate over collections.

Why

- Core to any logic; used for scoring rules and record transformations.

7. Batch Apex:

Purpose

Process large volumes of records asynchronously within governor limits.

Implementation

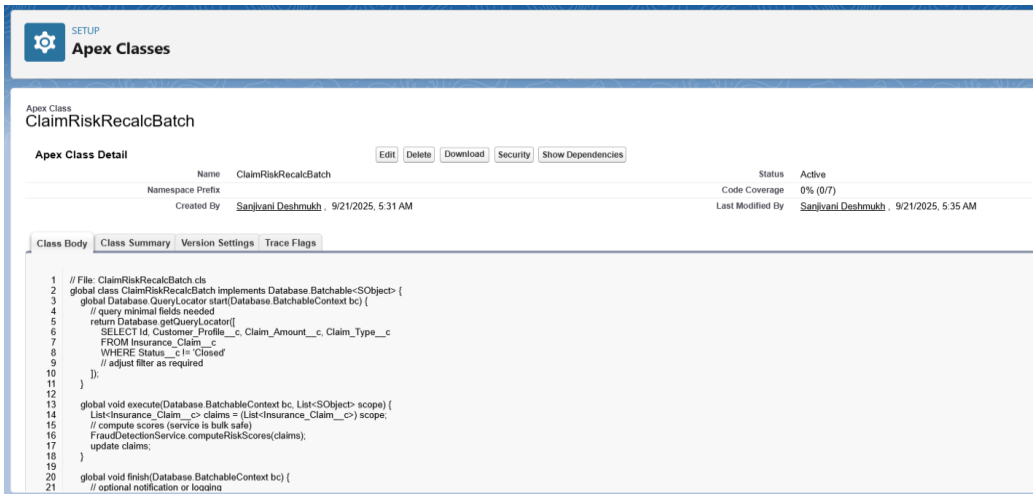
- **ClaimRiskRecalcBatch** — a **Database.Batchable<SObject>** implementation that re-calculates risk scores for claims in batches (chunk size e.g., 200).

Why

- If you have hundreds/thousands of claims, batch jobs let you recompute scores without hitting limits.

Implementation steps

1. Create batch class implementing start, execute, finish.
2. start returns a **Database.getQueryLocator()** with the claims to process.
3. execute calls **FraudDetectionService.computeRiskScores()** and updates chunk.
4. Schedule the batch via **System.schedule()** or a scheduled Apex class.



8. Queueable Apex:

Purpose

Asynchronous job for medium complexity tasks (chaining allowed) — lighter than Batch for smaller jobs.

Implementation

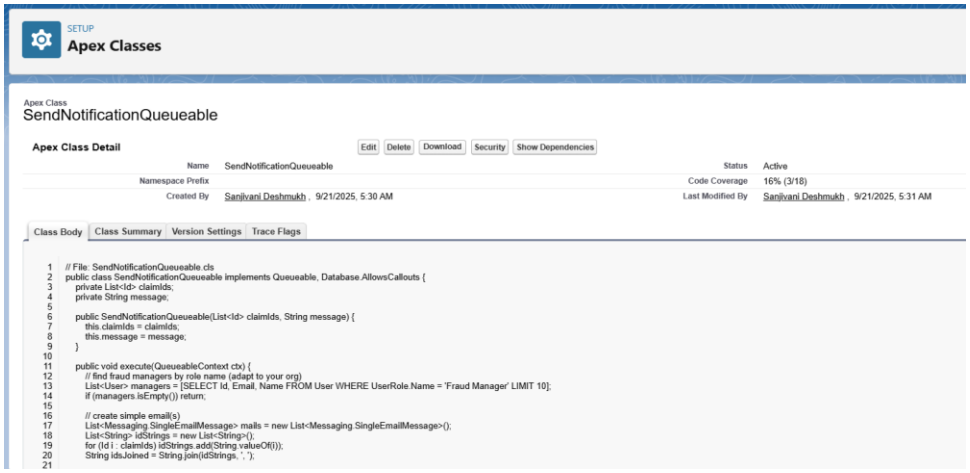
- **SendNotificationQueueable** to send emails or callouts asynchronously after fraud threshold reached.

Why

- Offloads email sending or callouts from trigger context to avoid long-running operations and limits.

Implementation notes

- Implement Queueable and call **System.enqueueJob(new SendNotificationQueueable(ids, message))**.
- If callouts are needed in queueable, implement **Database.AllowsCallouts.**



9. Scheduled Apex

Purpose

Run jobs on a schedule (daily/nightly).

Implementation

- **ClaimRiskRecalcScheduler** that executes **ClaimRiskRecalcBatch** nightly.

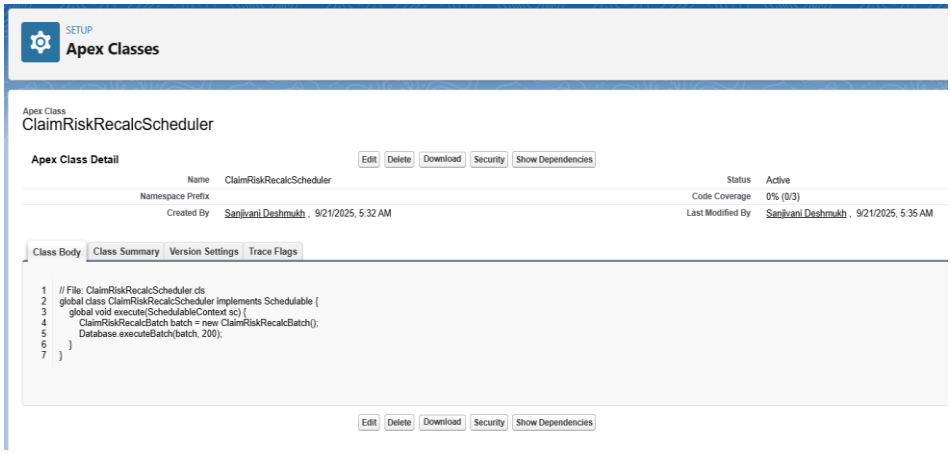
Why

- Regularly keep risk scores up-to-date and detect emerging fraud patterns.

How to schedule

- From Setup → Apex Classes → Schedule Apex, or use:

  System.schedule('Nightly Claim Risk', '0 0 2 * * ?', new ClaimRiskRecalcScheduler());



10. Future Methods

Purpose

Asynchronous execution for callouts that cannot be done synchronously in triggers.

Implementation

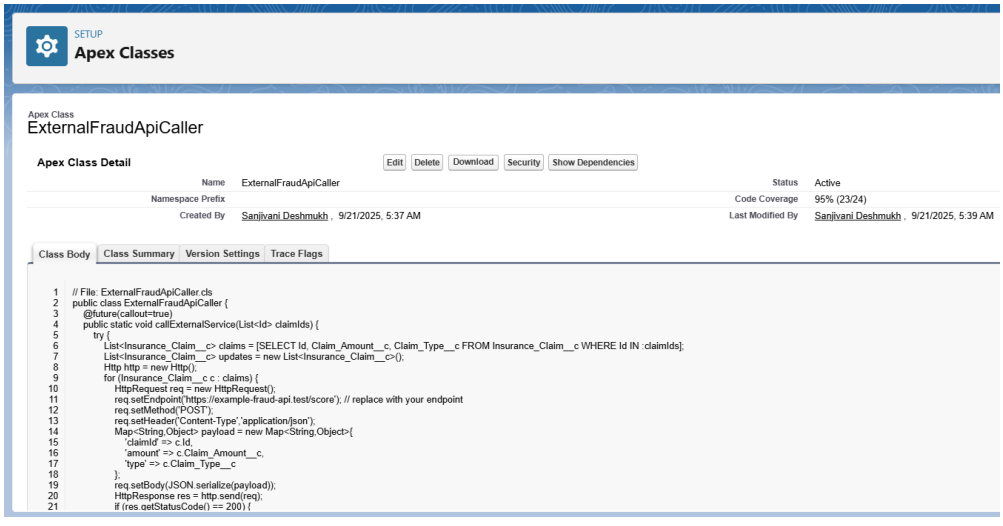
- **ExternalFraudApiCaller.callExternalService(List<Id> claimIds)** with **@future(callout=true)** as a wrapper to call an external fraud API and update scores.

Why

- Trigger context cannot perform callouts; **@future or Queueable** allows callouts asynchronously.

Important

- In tests, use **Test.setMock(HttpCalloutMock.class, ...)** to simulate external responses.



11. Exception Handling

Purpose
Catch and handle runtime errors without breaking user transactions unnecessarily.

Implementation

- try-catch blocks in callout and queueable classes; in triggers added **addError()** to block invalid saves when appropriate.

Why

- Provide graceful fallback (e.g., set default score if API fails) and log errors for troubleshooting.

12. Test Classes

Purpose
Unit tests validate logic, ensure >= 75% coverage and allow safe deployment.

Implementation

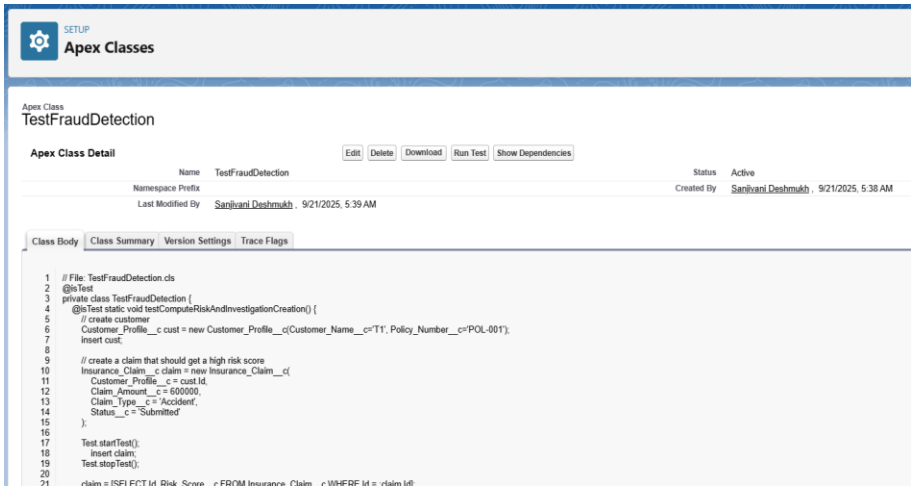
- **TestFraudDetection** test class with multiple test methods:
    - Insert customer & claim → assert **Risk_Score__c** computed and investigation created.
    - Insert **Fraud_Investigation__c** with **high Fraud_Score__c** → assert related claim status becomes In Review.
    - Mock HTTP callout and test **@future** call to external fraud API.

Why

- Required for deployment. Ensures code works across scenarios and handles async jobs.

Implementation notes

- Use **@isTest** annotation.
- Use **Test.startTest() / Test.stopTest()** around async operations to force execution within tests.
- Use **Test.setMock(HttpCalloutMock.class, new MockHttpResponseGenerator())** for callouts.



13. Asynchronous Processing (summary)

Purpose

Use Queueable, Batch, Scheduled Apex and Future to move heavy/remote/blocking work out of immediate transaction.

Used

- Queueable for notifications, Batch for nightly recalculation, Scheduled for running batch, Future for external API callouts.

Why

- Protects user experience, respects governor limits, supports scale.

Phase 6: User Interface Development

Goal:
Build a user-friendly and interactive Salesforce Lightning interface for Fraud Detection, enabling Adjusters, Analysts, and Managers to access claims, view fraud scores, and take actions seamlessly.

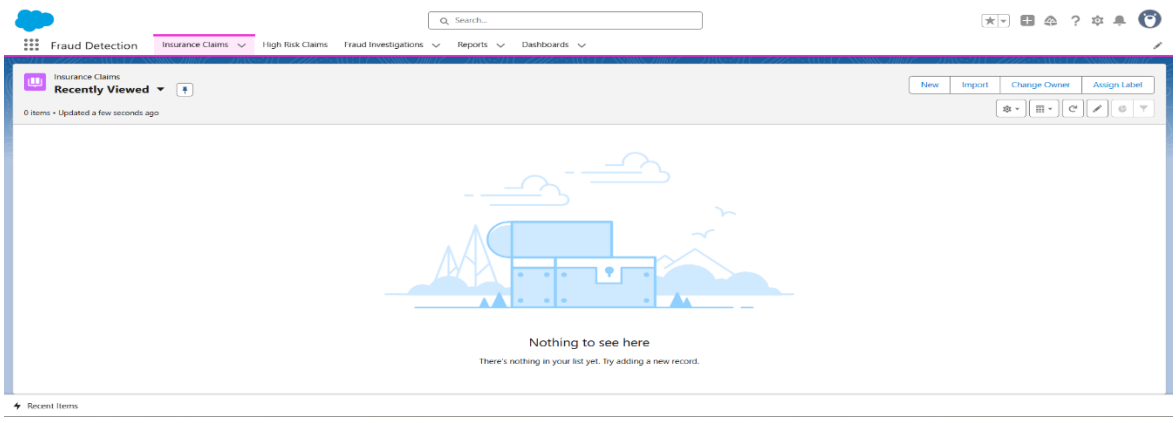
---

1. Lightning App Builder – Fraud Detection App

- Created a new **Lightning App** named ***Fraud Detection***.
- Added navigation items: **Insurance Claims**, **Fraud Investigations**, **Reports**, and **Dashboards**.
- Created **Custom Tabs** for custom objects and added them to the app.

- **Used:**
  - To give project stakeholders a **dedicated workspace** to manage claims and fraud investigations without navigating multiple apps.
- **Steps Followed:**

1. Setup → **App Manager** → **New Lightning App**.
2. Enter *App Name*: Fraud Detection.
3. Added navigation items (Insurance Claims, Fraud Investigations, Reports, Dashboards).
4. Assigned the app to **Analyst, Manager, and Adjuster profiles**.



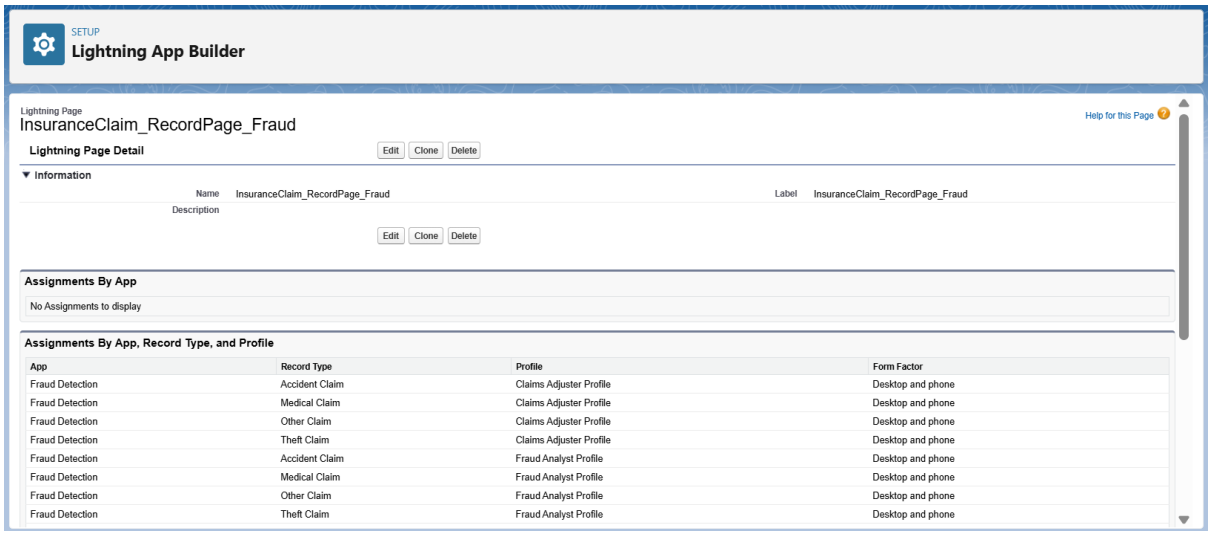
2. Record Pages – Claim Page Customization

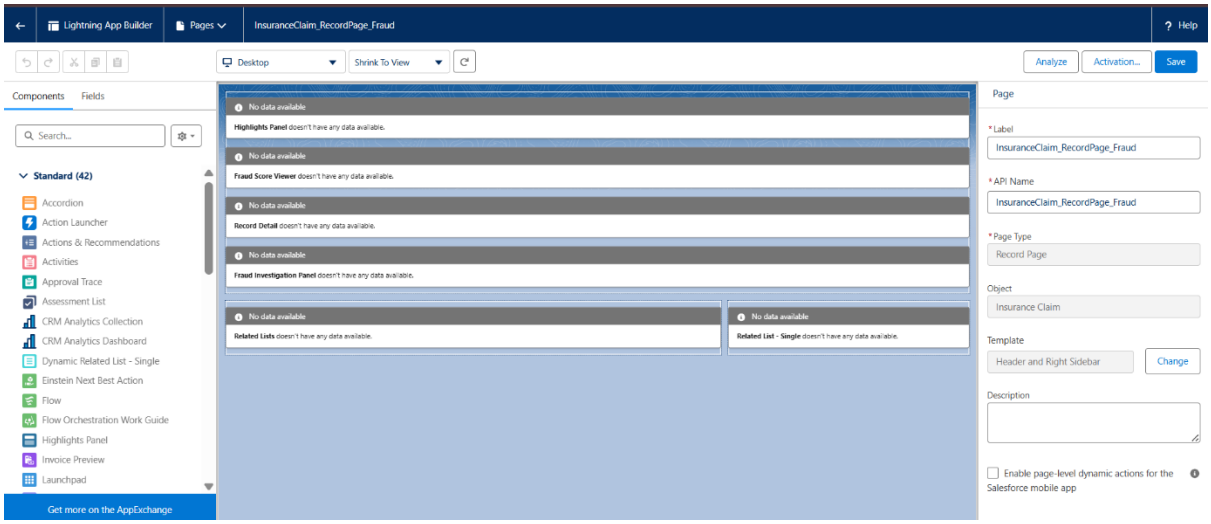
- Customized the **Insurance Claim record page** with components:
  - *Highlights Panel* (Claim #, Status, Risk Score).
  - *Record Detail* (claim details).
  - *Related Lists* (documents, fraud investigations).
  - *Custom LWCs* → fraudScoreViewer, fraudInvestigationPanel.

- **Used:**
  - To provide a **single view of claim and fraud details** for faster decision-making.

- **Steps Followed:**

1. Setup → **Lightning App Builder** → New Record Page.
2. Selected **Insurance Claim** object.
3. Dragged required components into layout.
4. Added custom LWCs after deployment.
5. Activated the page for the *Fraud Detection App*.





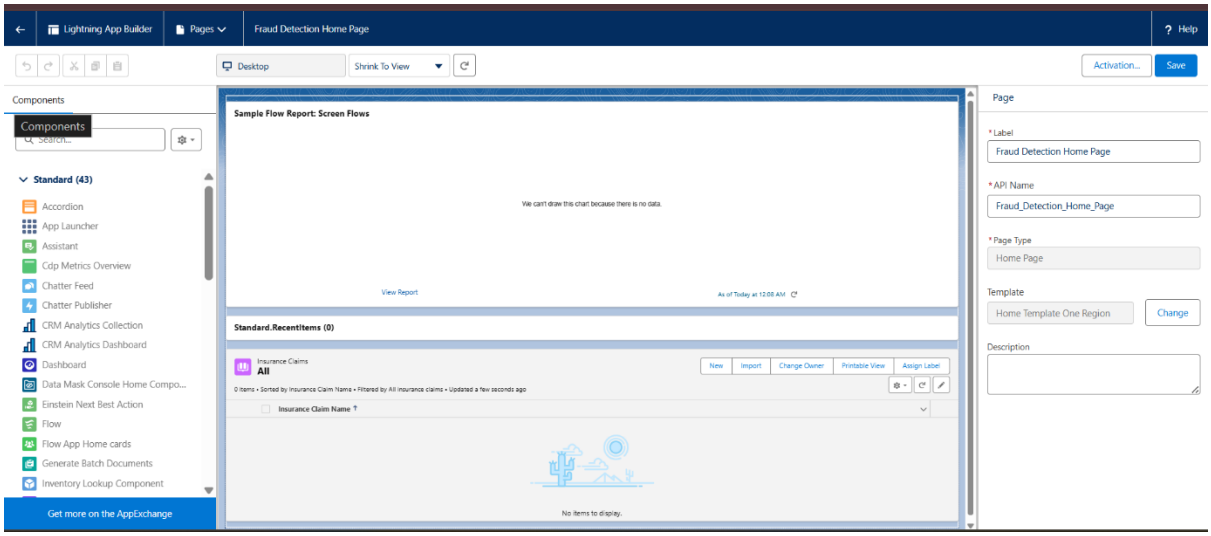
---

3. Home Page Layouts:

- Created a Fraud Detection Home Page.
- Added components:
  - *Report Chart* → Fraud Trends.
  - *Recent Items*.
  - *List View* → High Risk Claims.

- **Used:**
  - To give Managers a dashboard-like overview of fraud risks and recent claims.
- **Steps Followed:**

1. Setup → Lightning App Builder → New Home Page.
2. Added components: *Report Chart, Recent Items, List View*.
3. Activated for the *Fraud Detection App* and profiles.



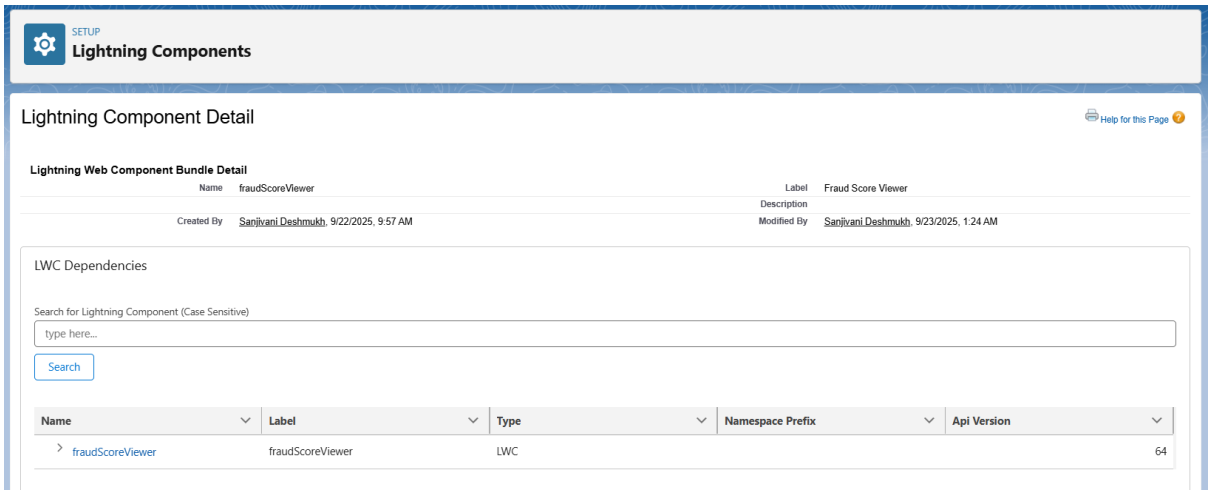
---

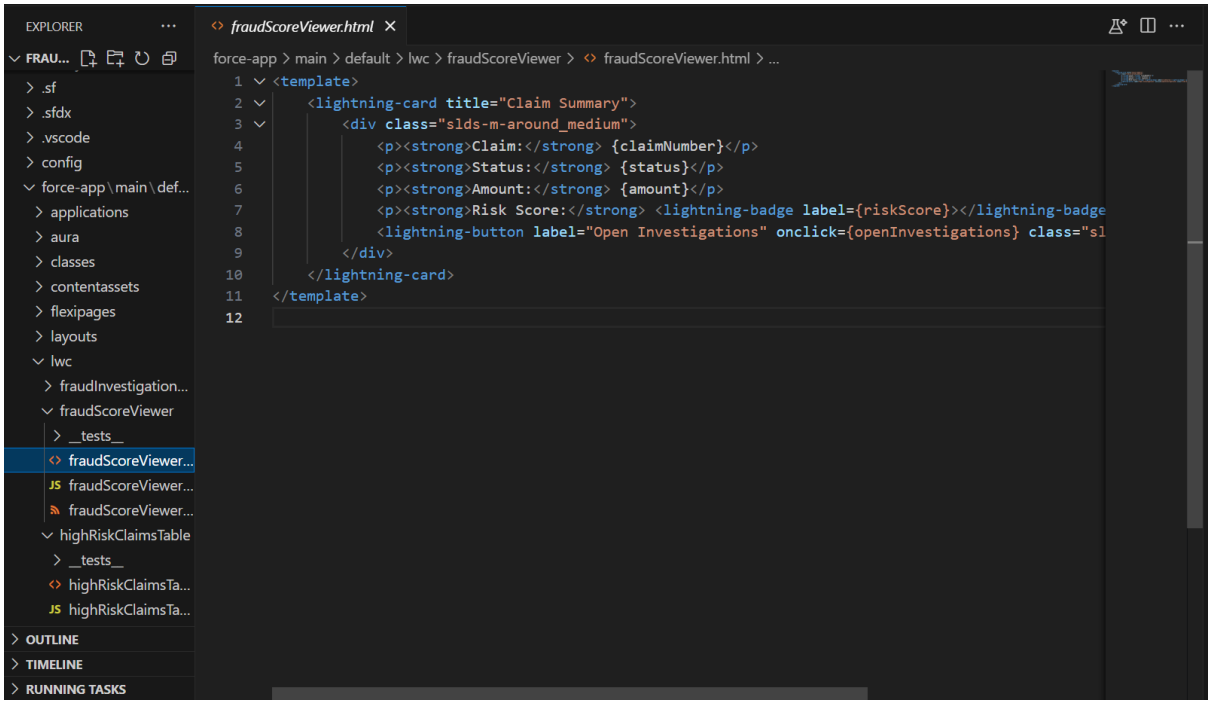
4. Lightning Web Components (LWCs)

Developed 3 LWCs to extend UI functionality:

(a) fraudScoreViewer

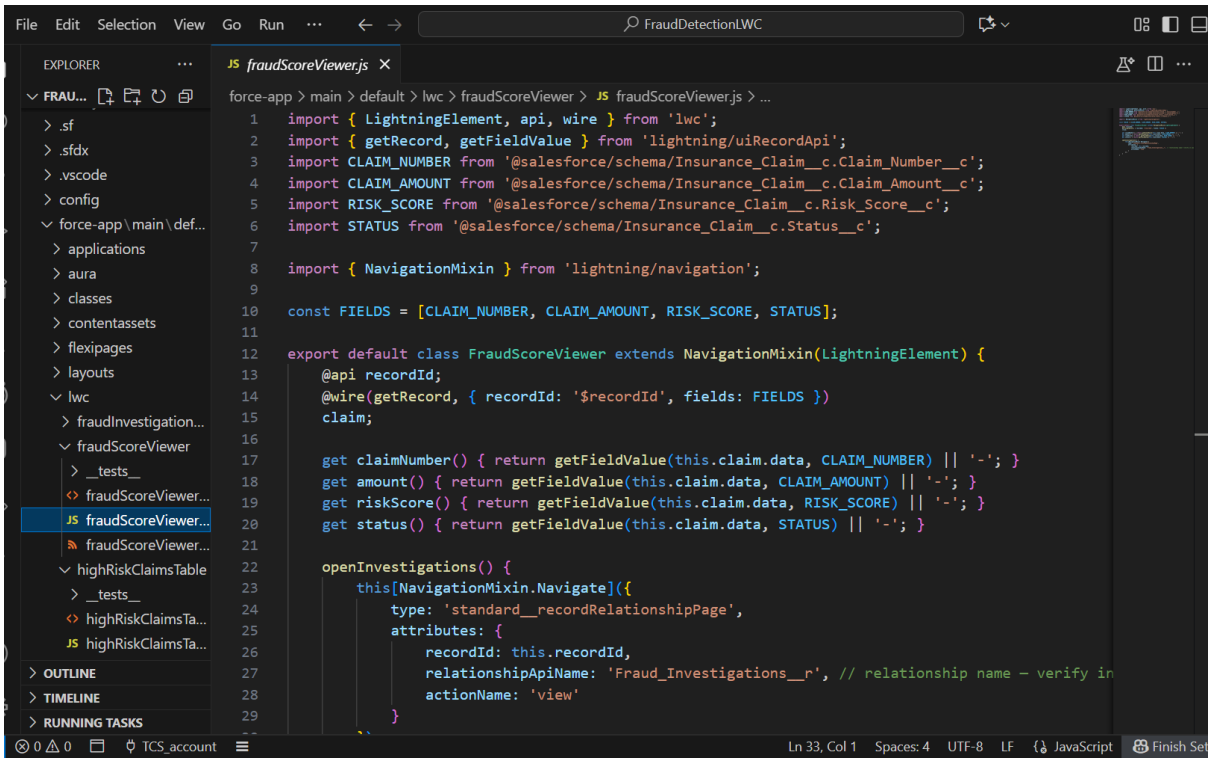
- **Purpose:** Displays real-time fraud score for a claim.
- **Usage:** Added to Insurance Claim Record Page.
- **Why:** Allows Analysts to instantly check fraud risk without running reports.

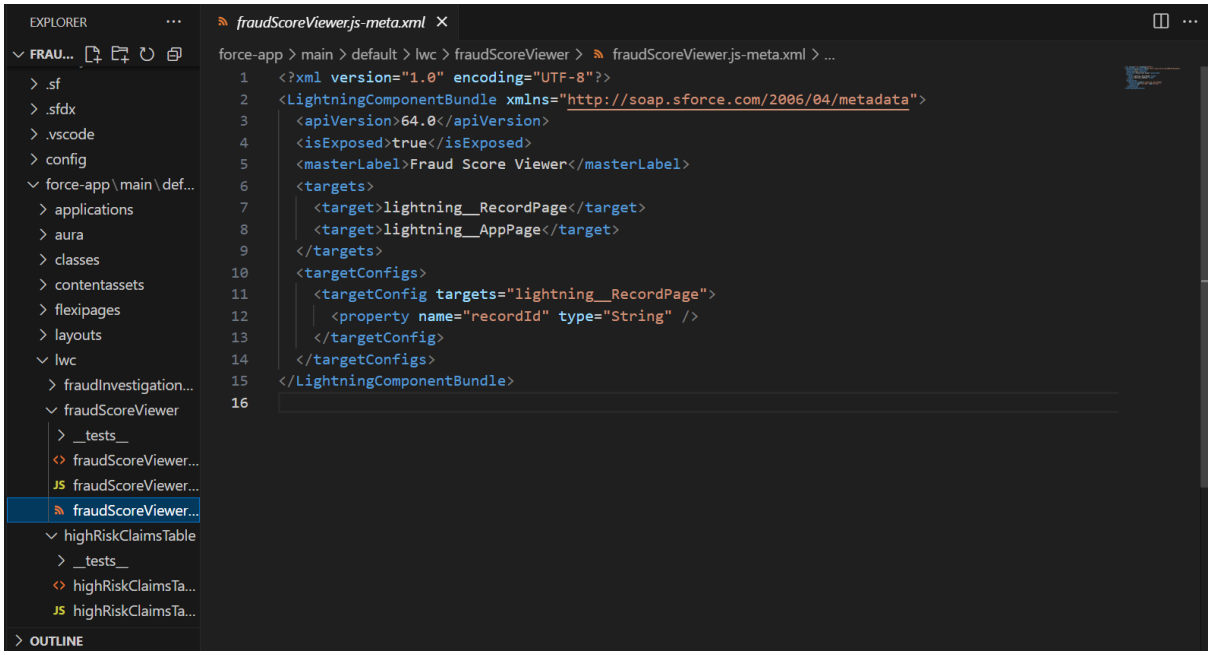



EXPLORER
fraudScoreViewer.html
force-app > main > default > lwc > fraudScoreViewer > fraudScoreViewer.html > ...
<template>
<lightning-card title="Claim Summary">

Claim: {claimNumber}
Status: {status}
Amount: {amount}
Risk Score: <lightning-badge label={riskScore}></lightning-badge
<lightning-button label="Open Investigations" onclick={openInvestigations} class="sl

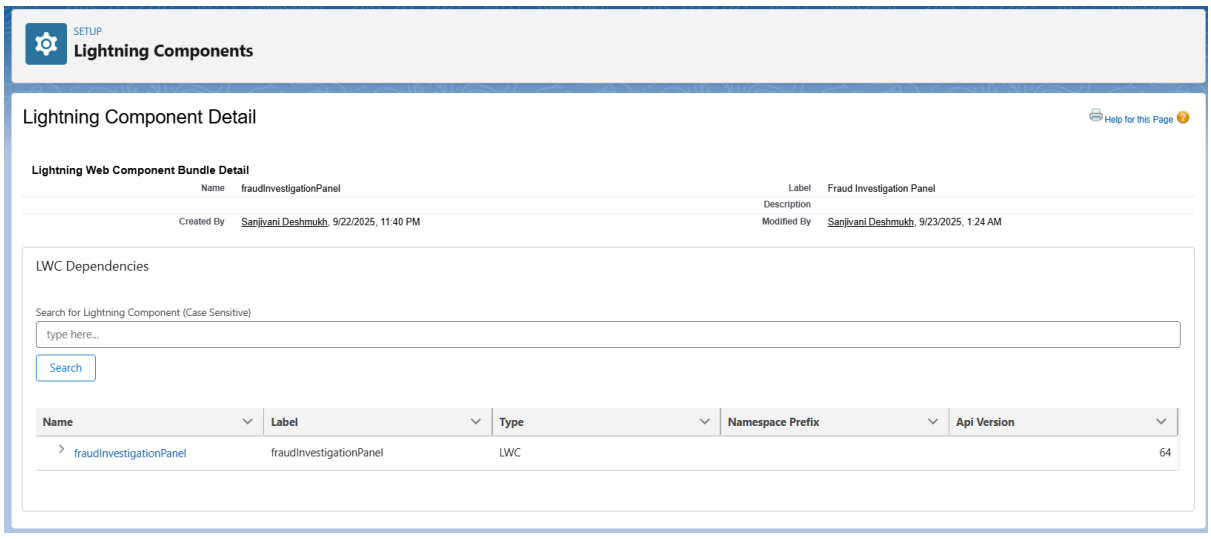
</lightning-card>
</template>
OUTLINE
TIMELINE
RUNNING TASKS

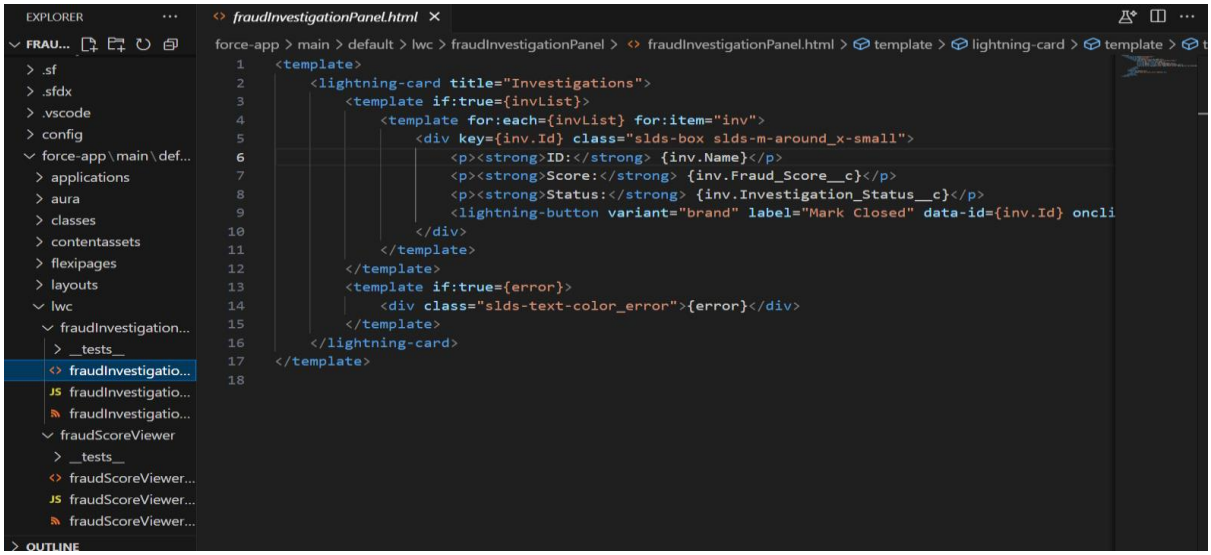

File Edit Selection View Go Run
FraudDetectionLWC
EXPLORER
fraudScoreViewer.js
force-app > main > default > lwc > fraudScoreViewer > fraudScoreViewer.js > ...
import { LightningElement, api, wire } from 'lwc';
import { getRecord, getFieldValue } from 'lightning/uiRecordApi';
import CLAIM_NUMBER from '@salesforce/schema/Insurance_Claim__c.Claim_Number__c';
import CLAIM_AMOUNT from '@salesforce/schema/Insurance_Claim__c.Claim_Amount__c';
import RISK_SCORE from '@salesforce/schema/Insurance_Claim__c.Risk_Score__c';
import STATUS from '@salesforce/schema/Insurance_Claim__c.Status__c';
import { NavigationMixin } from 'lightning/navigation';
const FIELDS = [CLAIM_NUMBER, CLAIM_AMOUNT, RISK_SCORE, STATUS];
export default class FraudScoreViewer extends NavigationMixin(LightningElement) {
@api recordId;
@wire(getRecord, { recordId: '$recordId', fields: FIELDS })
claim;
get claimNumber() { return getFieldValue(this.claim.data, CLAIM_NUMBER) || '-'; }
get amount() { return getFieldValue(this.claim.data, CLAIM_AMOUNT) || '-'; }
get riskScore() { return getFieldValue(this.claim.data, RISK_SCORE) || '-'; }
get status() { return getFieldValue(this.claim.data, STATUS) || '-'; }
openInvestigations() {
this[NavigationMixin.Navigate]({
type: 'standard__recordRelationshipPage',
attributes: {
recordId: this.recordId,
relationshipApiName: 'Fraud_Investigations__r', // relationship name – verify in
actionName: 'view'
}
OUTLINE
TIMELINE
RUNNING TASKS
TCS_account
Ln 33, Col 1
Spaces: 4
UTF-8
LF
JavaScript



(b) fraudInvestigationPanel

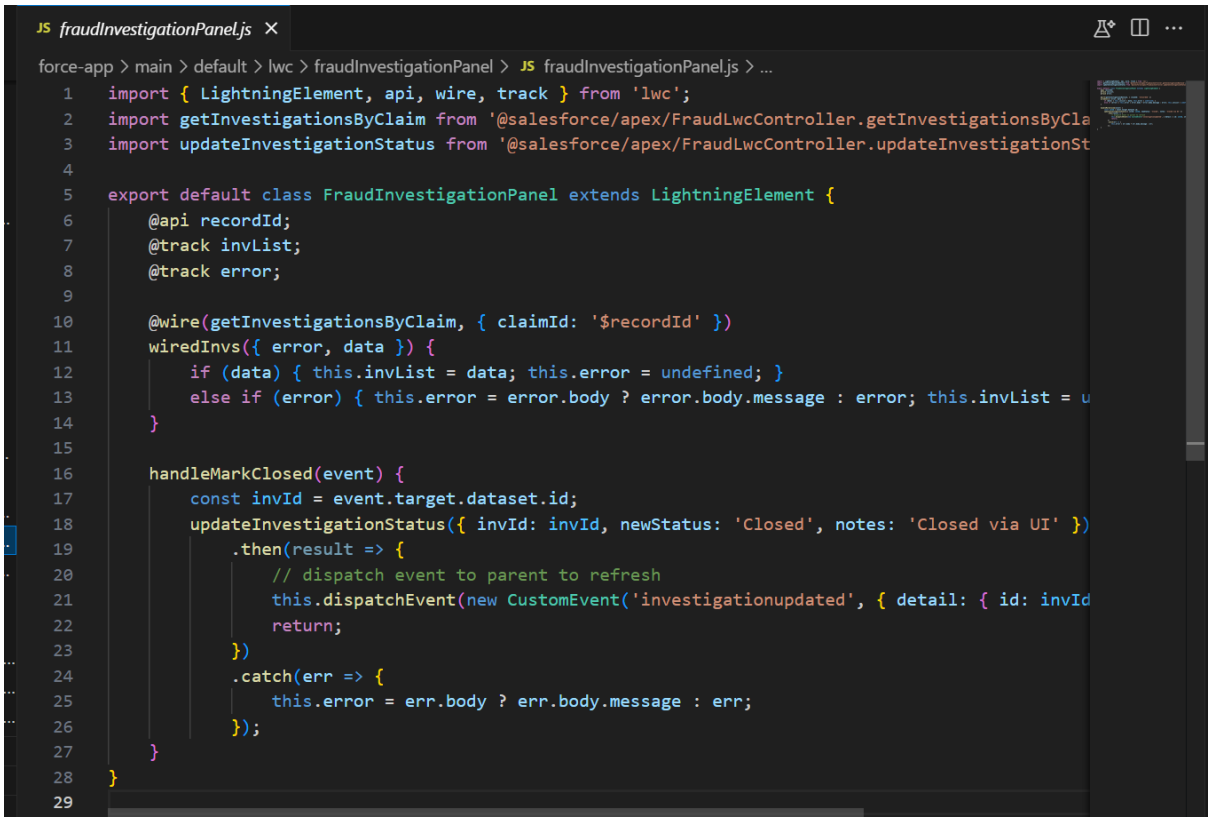
- **Purpose:** Shows related investigations for a claim, with option to create or update investigations.
- **Usage:** Added to record page layout.
- **Why:** Helps Analysts track fraud investigations directly from the claim page.




EXPLORER
fraudInvestigationPanel.html
force-app > main > default > lwc > fraudInvestigationPanel > fraudInvestigationPanel.html > template > lightning-card > template
<template>
<lightning-card title="Investigations">
<template if:true={invList}>
<template for:each={invList} for:item="inv">

ID: {inv.Name}
Score: {inv.Fraud_Score__c}
Status: {inv.Investigation_Status__c}
<lightning-button variant="brand" label="Mark Closed" data-id={inv.Id} oncli

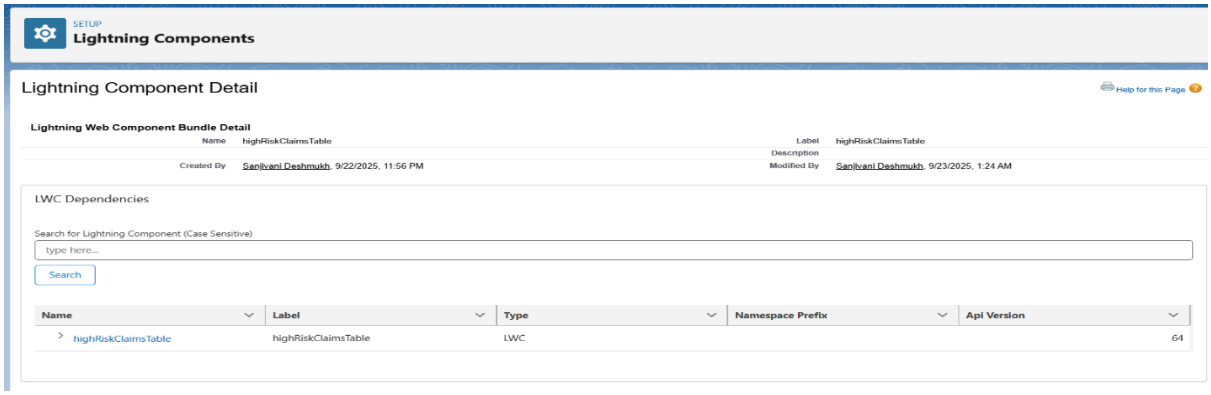
</template>
</template>
<template if:true={error}>
{error}
</template>
</lightning-card>
</template>
.sf
.sfdx
.vscode
config
force-app\main\def...
applications
aura
classes
contentassets
flexipages
layouts
lwc
fraudInvestigation...
__tests__
fraudInvestigatio...
fraudInvestigatio...
fraudInvestigatio...
fraudScoreViewer
__tests__
fraudScoreViewer...
fraudScoreViewer...
fraudScoreViewer...
OUTLINE

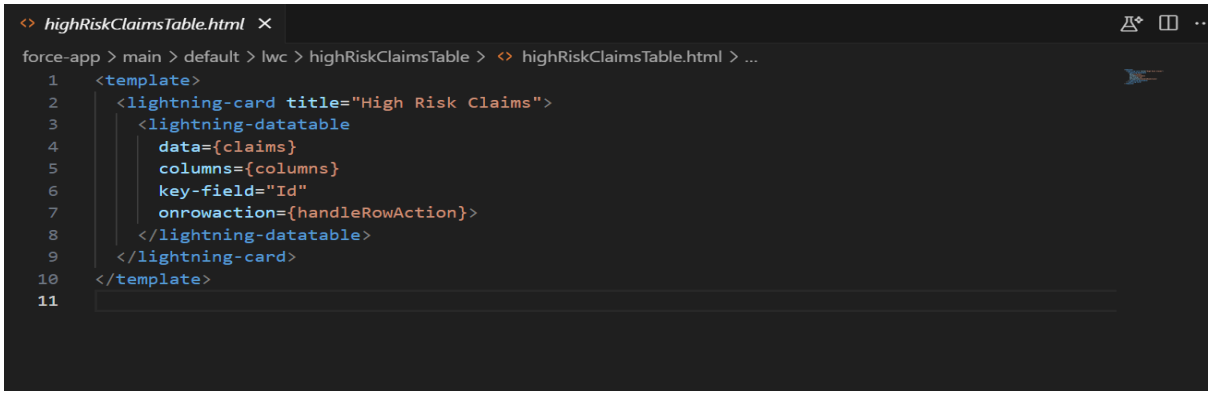

fraudInvestigationPanel.js
force-app > main > default > lwc > fraudInvestigationPanel > fraudInvestigationPanel.js > ...
import { LightningElement, api, wire, track } from 'lwc';
import getInvestigationsByClaim from '@salesforce/apex/FraudLwcController.getInvestigationsByCla
import updateInvestigationStatus from '@salesforce/apex/FraudLwcController.updateInvestigationSt
export default class FraudInvestigationPanel extends LightningElement {
@api recordId;
@track invList;
@track error;
@wire(getInvestigationsByClaim, { claimId: '$recordId' })
wiredInvs({ error, data }) {
if (data) { this.invList = data; this.error = undefined; }
else if (error) { this.error = error.body ? error.body.message : error; this.invList = u
}
handleMarkClosed(event) {
const invId = event.target.dataset.id;
updateInvestigationStatus({ invId: invId, newStatus: 'Closed', notes: 'Closed via UI' })
.then(result => {
// dispatch event to parent to refresh
this.dispatchEvent(new CustomEvent('investigationupdated', { detail: { id: invId
return;
})
.catch(err => {
this.error = err.body ? err.body.message : err;
});
}
}


fraudInvestigationPanel.js-meta.xml
force-app > main > default > lwc > fraudInvestigationPanel > fraudInvestigationPanel.js-meta.xml > ...
<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
<apiVersion>64.0</apiVersion>
<isExposed>true</isExposed>
<masterLabel>Fraud Investigation Panel</masterLabel>
<targets>
<target>lightning__RecordPage</target>
</targets>
</LightningComponentBundle>

(c) highRiskClaimsTable

- **Purpose:** Displays all high-risk claims in a table view.
- **Usage:** Added to Home Page or Utility Bar.
- **Why:** Allows Managers to monitor all high-risk claims at a glance.





highRiskClaimsTable.js-meta.xml

force-app > main > default > lwc > highRiskClaimsTable > highRiskClaimsTable.js-meta.xml > ...

```
<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>64.0</apiVersion>
    <isExposed>true</isExposed>
    <targets>
        <!-- Can be added on App Pages -->
        <target>lightning__AppPage</target>

        <!-- Can be added on Home Page -->
        <target>lightning__HomePage</target>

        <!-- Can be added on Record Pages -->
        <target>lightning__RecordPage</target>

        <!-- Can be added in Utility Bar -->
        <target>lightning__UtilityBar</target>
    </targets>
</LightningComponentBundle>

```

---

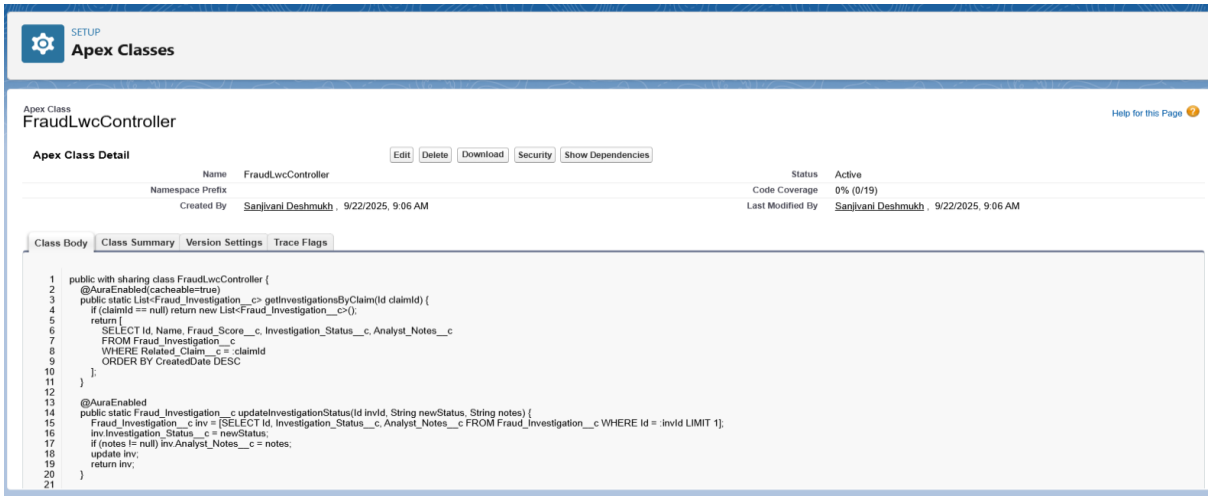
5. Apex with LWC Integration

- Created Apex Controllers (**FraudLwcController**) to fetch claim and investigation data.
- Granted Apex Class Access to Analyst and Manager profiles.

- **Used:**
  - LWC cannot directly access Salesforce objects beyond Lightning Data Service; Apex ensures complex queries and business logic can be handled.

- **Steps:**

1. Setup → Apex Classes → Created controllers.
2. Setup → Profiles → Added Apex Class Access.



---

6. Events in LWC

- Used Custom Events (**investigationupdated**) in **fraudInvestigationPanel** to notify parent components.

- **Used:**
  - Enables real-time refresh of data without reloading the page.

---

7. Wire Adapters & Imperative Calls

- **Wire Adapters:**
  - Used **@wire** with Apex methods to fetch claim and investigation data.
  - Why: Automatic refresh when underlying data changes.
- **Imperative Calls:**
  - Used for on-demand operations like creating investigations.
  - Why: Offers flexibility when updates should only happen after user action.

---

8. Navigation Service

- Used **NavigationMixin.Navigate** to navigate from **highRiskClaimsTable** to claim record page.

- **Used:**
  - Improves usability, allowing users to jump directly to claim details.

---

9. Permissions & Profiles

    - Granted Apex Class Access and LWC access to Analyst and Manager profiles.
    - Mapped Lightning Record Pages and Home Page to Fraud Detection App.
- **Used:**
    - To ensure only authorized users (Analyst/Manager) can access fraud-related functionality.

Phase 7: Integration & External Access

Goal:

Enable Salesforce to securely communicate with external systems (fraud detection API, other services) while ensuring data security, scalability, and compliance with Salesforce governor limits.

---

1. Named Credentials (✔ Used)

Purpose:

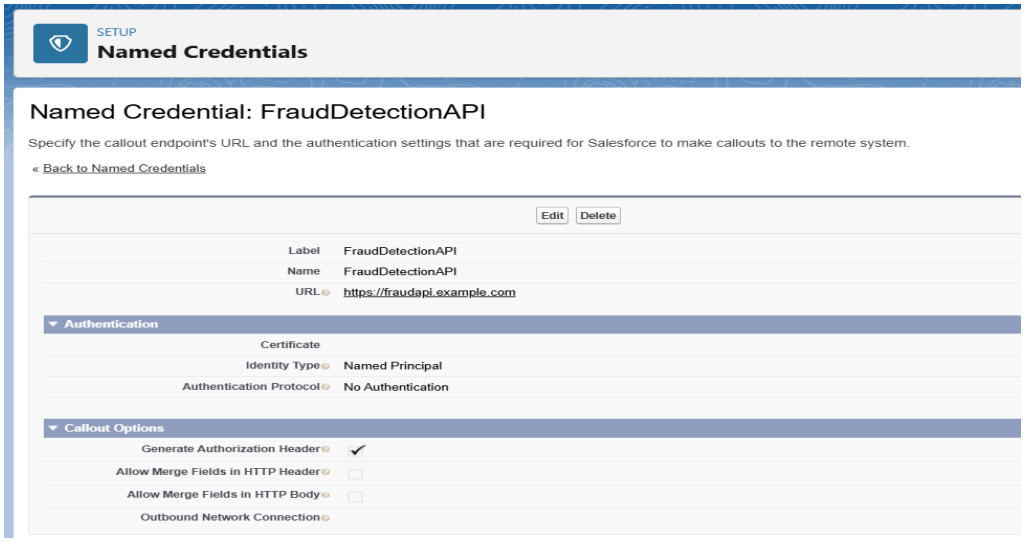
Securely store the base URL and authentication settings for an external API without hardcoding credentials inside Apex classes.

Steps Followed:

1. Setup → Named Credentials → New.
2. Added **Label**: **FraudAPI** and **URL**: **https://fraudapi.example.com.**
3. Chose **Identity Type: Anonymous** and **Authentication Protocol: No Authentication**, since using a demo API without login credentials.
4. Saved the configuration.

Why Important?

- Avoids hardcoding API endpoints in Apex.
- Centralized management of authentication.
- Secure and easy to update if endpoint changes.



---

2. External Services (❌ Not Used)

Purpose:

Allows Salesforce to consume APIs described by an OpenAPI (Swagger) specification without writing code. Salesforce auto-generates Apex actions from the service schema.

Reason for Skipping:

- Our demo fraud detection API doesn't provide Swagger/OpenAPI definitions.
- Instead, we directly integrated via Named Credentials + Apex Callout.

---

3. Web Services (REST/SOAP) (✔ Used – REST)

Purpose:

Salesforce supports integrating with external systems via REST or SOAP APIs.

- REST → lightweight, JSON-based (used in our project).
- SOAP → XML-based, older format (not used here).

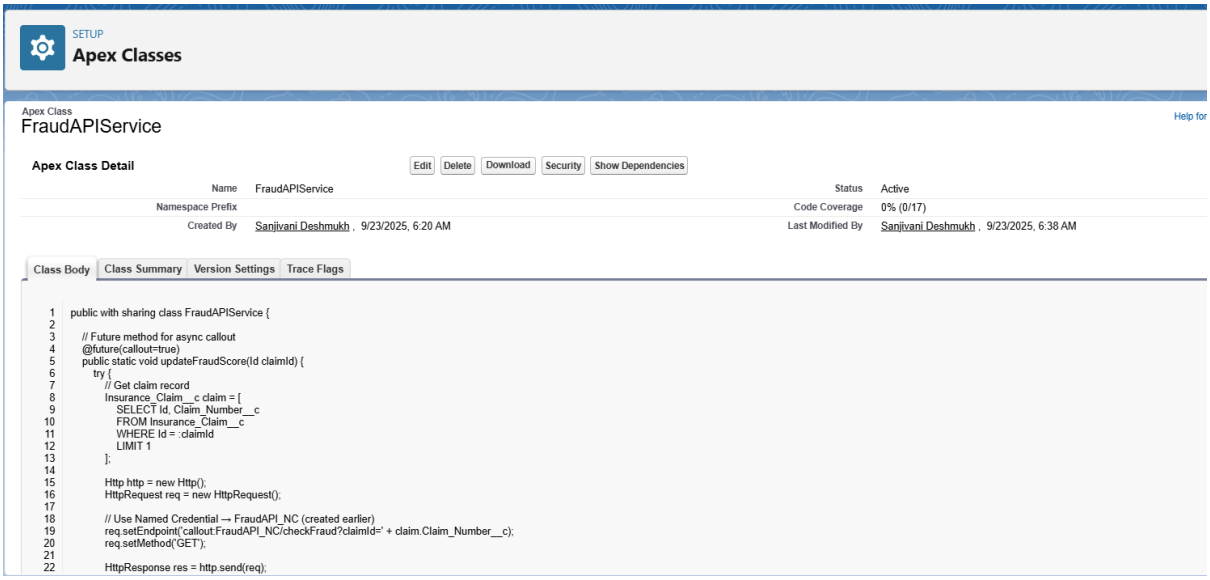
Implementation:

- Created an Apex class with @future(callout=true) to send REST callouts.
- Example: Fetch fraud score for a claim using **req.setEndpoint('callout:FraudAPI/score').**

Code:

```
public with sharing class FraudAPIService {

  // Future method for async callout
  @future(callout=true)
  public static void updateFraudScore(Id claimId) {
    try {
      // Get claim record
      Insurance_Claim__c claim = [
        SELECT Id, Claim_Number__c
        FROM Insurance_Claim__c
        WHERE Id = :claimId
        LIMIT 1
      ];
```

```
        Http http = new Http();
        HttpRequest req = new HttpRequest();

        // Use Named Credential → FraudAPI_NC (created earlier)
        req.setEndpoint('callout:FraudAPI_NC/checkFraud?claimId=' + claim.Claim_Number_
_c);
        req.setMethod('GET');

        HttpResponse res = http.send(req);

        if (res.getStatusCode() == 200) {
            // Parse JSON response
            Map<String, Object> result =
                (Map<String, Object>) JSON.deserializeUntyped(res.getBody());

            if (result.containsKey('fraudScore')) {
                Decimal score = Decimal.valueOf(result.get('fraudScore').toString());

                // Update claim with fraud score
                claim.Risk_Score__c = score;
                update claim;
            }
        } else {
            System.debug('Fraud API Callout failed → ' + res.getStatusCode() + ' : ' + res.getBod
y());
        }
    } catch (Exception e) {
        System.debug('Error in FraudAPIService.updateFraudScore: ' + e.getMessage());
    }
  }
}
```



---

4. Callouts (✔ Used)

Purpose:

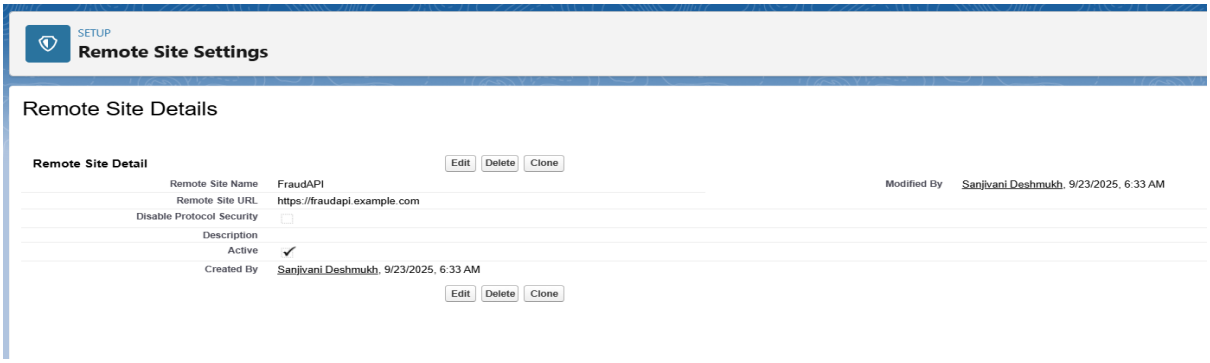
Allow Salesforce to send HTTP requests (GET/POST) to external systems.

Steps:

1. Added Named Credential → FraudAPI.
2. Wrote Apex code:
3. HttpRequest req = new HttpRequest();
4. req.setEndpoint('callout:FraudAPI/score');
5. req.setMethod('GET');
6. HttpResponse res = new Http().send(req);
7. Parsed JSON response and updated Claim Risk Score.

Why Important?

- Enables Salesforce to interact with real-time fraud detection services.
- Provides fraud score enrichment for Insurance Claims.



---

5. Platform Events (✔ Used)

Purpose:

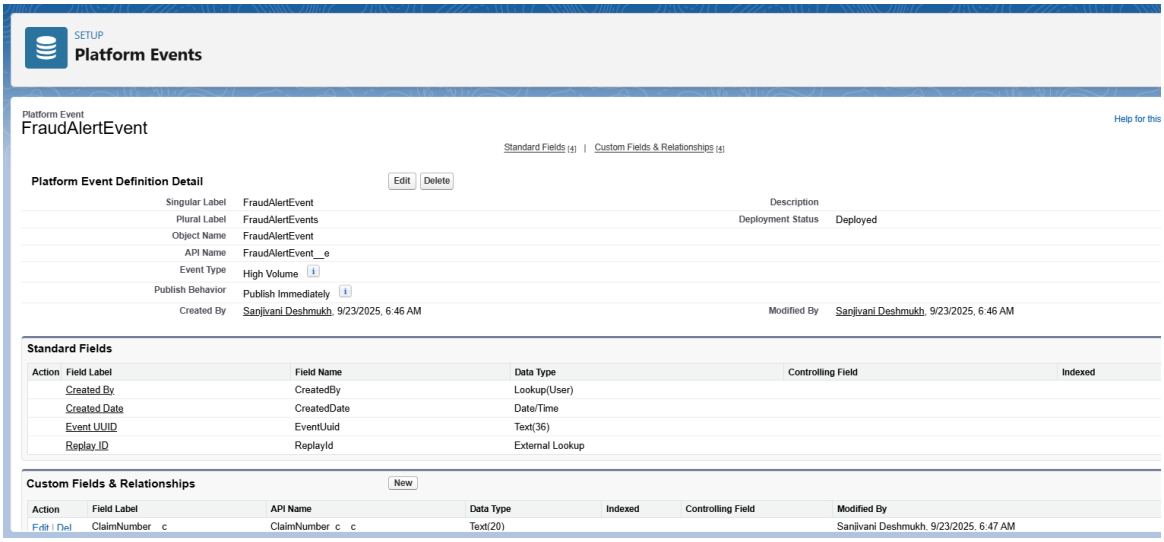
Enable event-driven architecture for real-time communication inside Salesforce and with external systems.

Example in Our Project:

- Created a Fraud_Alert__e platform event.
- Published event when claim fraud score > 80.
- External listeners can react to fraud alerts immediately.

Why Important?

- Decouples fraud detection logic from notifications.
- Supports real-time alerting and system integration.



File ▾ Edit ▾ Debug ▾ Test ▾ Workspace ▾ Help ▾ < >

FraudAlertPublisher.apxc

Code Coverage: None ▾ API Version: 64

```apex
public class FraudAlertPublisher {

    // Method to publish the Fraud Alert Event
    public static void publishFraudAlert(String claimNumber, Decimal fraudScore, String customerName) {

        // Create an instance of your Platform Event
        FraudAlertEvent__e alert = new FraudAlertEvent__e(
            ClaimNumber_c__c = claimNumber,
            FraudScore_c__c = fraudScore,
            CustomerName_c__c = customerName,
            Message_c__c = 'High-risk claim detected'
        );

        // Publish the event
        Database.SaveResult result = EventBus.publish(alert);

        if (result.isSuccess()) {
            System.debug('Fraud Alert Event Published Successfully: ' + claimNumber);
        } else {
            System.debug('Failed to publish event: ' + result.getErrors()[0].getMessage());
        }
    }
}
```

SETUP

Apex Triggers

Apex Trigger

FraudAlertEventTrigger

Apex Trigger Detail Edit | Delete | Download | Show Dependencies

Name	FraudAlertEventTrigger	sObject Type	FraudAlertEvent
Code Coverage	0% (0/1)	Status	Active
Created By	Sanjivani Deshmukh, 9/23/2025, 7:07 AM	Last Modified By	Sanjivani Deshmukh, 9/23/2025, 7:13 AM
Namespace Prefix			

Apex Trigger | Version Settings | Trace Flags

```apex
trigger FraudAlertEventTrigger on FraudAlertEvent__e (after insert) {
    for(FraudAlertEvent__e eventRecord : Trigger.New) {
        System.debug('Fraud Alert for Claim: ' + eventRecord.ClaimNumber_c__c);
        // Call external system via Apex callout if needed
    }
}
```

Edit | Delete | Download | Show Dependencies

---

6. Change Data Capture (❌ Not Used)

Purpose:

Automatically publishes events whenever records in selected objects change.

Reason for Skipping:

- Not essential for our demo project.
- Could be used in future for syncing Claim/Investigation updates with external databases.

---

7. Salesforce Connect (❌ Not Used)

Purpose:

Connects external databases (like Oracle, SAP, AWS) as virtual objects in Salesforce without storing data locally.

Reason for Skipping:

- Our fraud detection service provides API access only, not database connection.
- Salesforce Connect not required.

---

8. API Limits

Purpose:

Salesforce imposes limits to ensure system performance.

Important Limits:

- **REST API Calls per 24 hours** → Based on license (e.g., 15,000 for Enterprise Edition).
- **Callout timeout** → 120 seconds max.
- **Concurrent callouts** → Up to 10 per Apex transaction.

Why Important?

- Helps us design integrations without exceeding limits.
- In real projects, ensures API quota is not exhausted.

---

9. OAuth & Authentication (❌ Not Used – Chose No Auth)

Purpose:

OAuth2 is a standard authentication method for connecting Salesforce to third-party services.

Why Skipped:

- Our fraud detection API is a demo and doesn't require credentials.
- Chose **No Authentication** in Named Credentials.

Note:

In a real system, APIs usually require OAuth2 / Password Authentication for security.

10. Remote Site Settings (✔ Used)

Purpose:

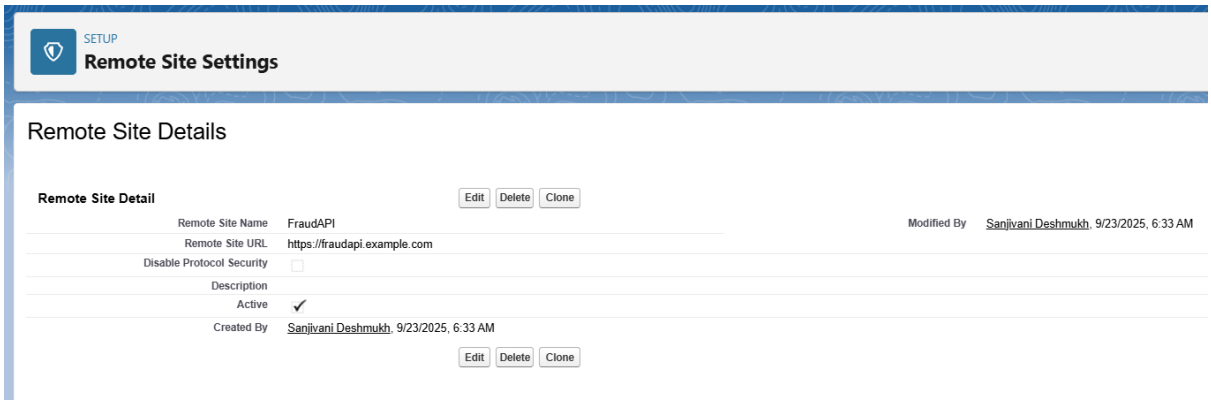
Allow Salesforce to call external services securely.

Steps:

- For Named Credentials → Remote Site Settings are automatically handled.
- If calling endpoint directly, you must manually add base URL in **Setup → Remote Site Settings**.

Why Important?

- Prevents unauthorized external callouts.
- Adds a layer of security by whitelisting allowed domains.



Phase 8: Data Management & Deployment

🎯 **Goal**: Manage financial transaction data efficiently and move project configurations between environments.

---

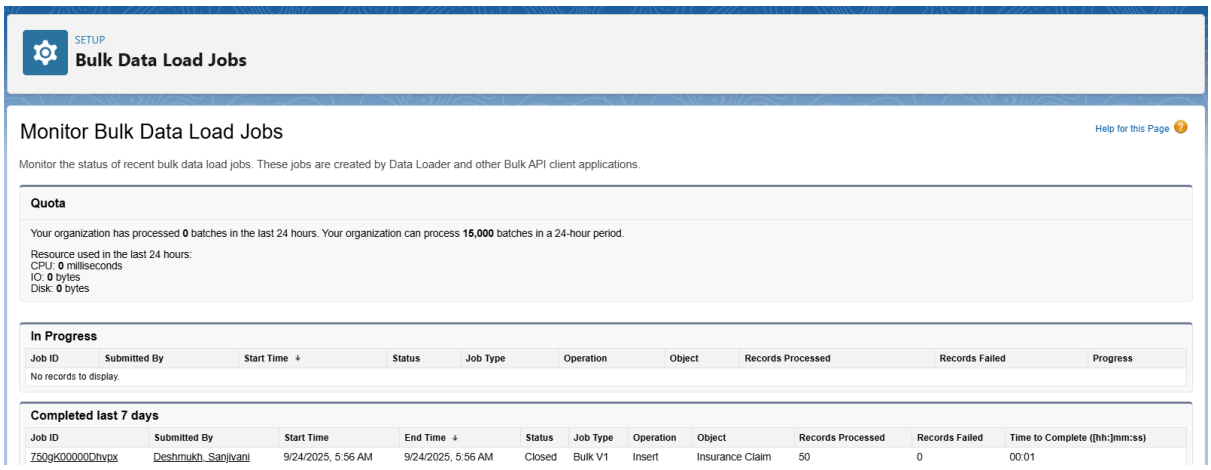
1. Data Import Wizard:

Purpose: Import small demo datasets into Salesforce for testing.

Steps Followed:

- Created a sample CSV with ~50 **demo financial transactions** (Transaction ID, Customer ID, Amount, Location, Risk Score, Fraud Flag).
- Used **Data Import Wizard** (Setup → Data Import Wizard → Custom Objects → Transaction).
- Imported the records successfully (Claim/Transaction ID auto-generated if using Auto Number).

Why Important?
Provides test data for fraud monitoring flows, validations, and dashboards.



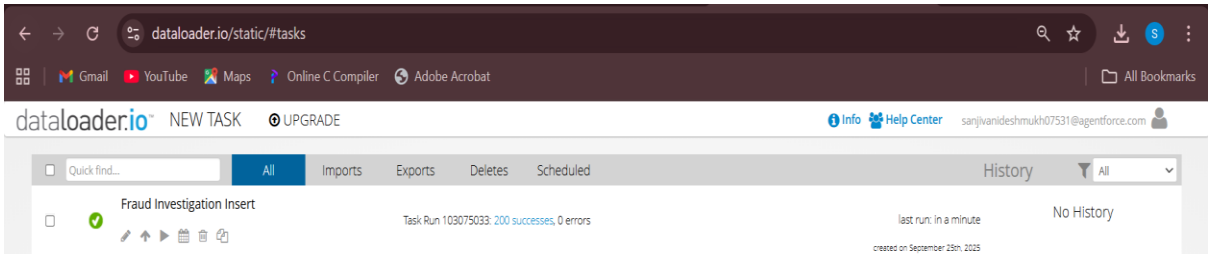
---

2. Data Loader (Bulk Data):

Purpose: Import or export large datasets (50k+ records).

- Can be used if you want to test system performance with **bulk financial transactions**.
- Supports insert, update, upsert, delete.

Why Important?
Useful for **stress testing fraud detection algorithms** with bigger data.



---

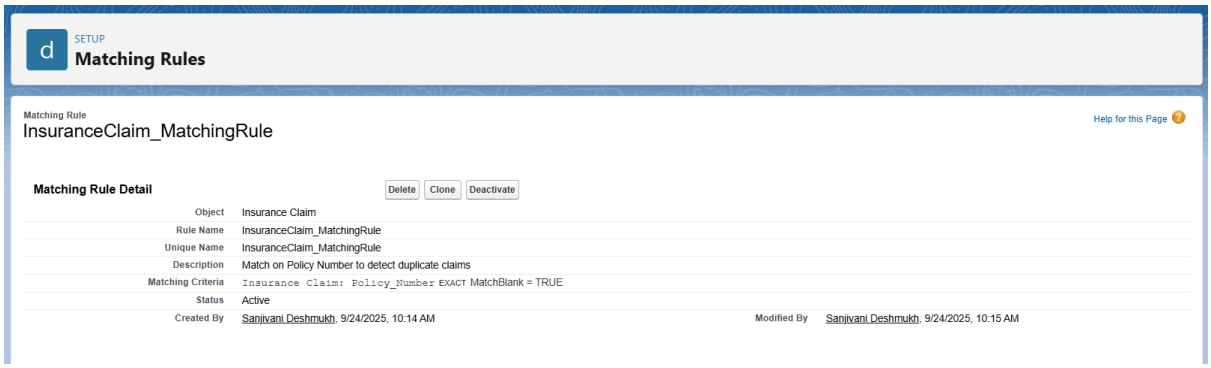
3. Duplicate Rules:

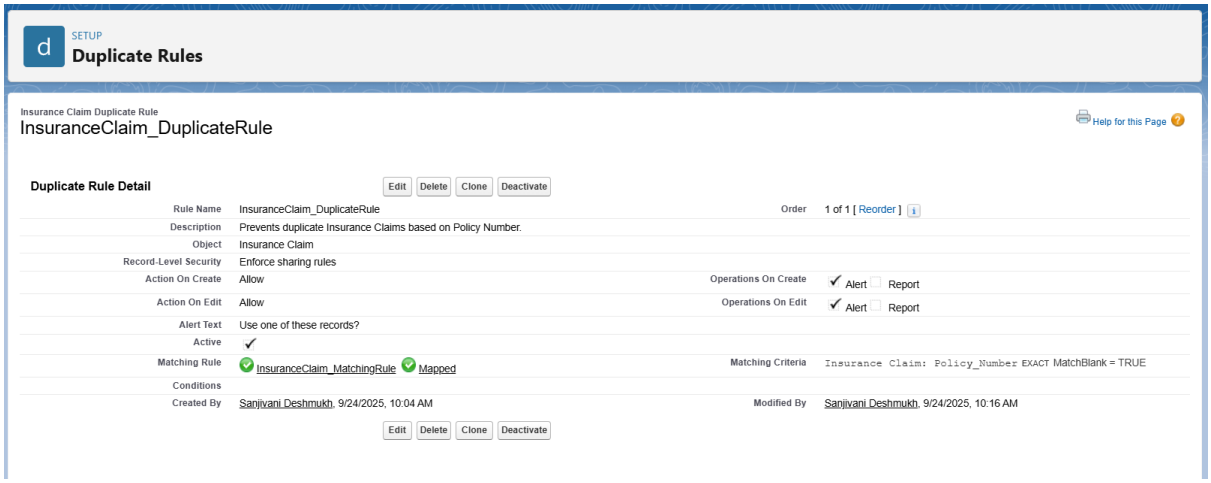
Purpose: Prevent duplicate **transaction records or customer profiles**.

Steps:

- Setup → Duplicate Rules → New Rule.
- For **Customer Profile**, match on Email/Phone.
- For **Transactions**, match on Transaction ID.

Why Important?
Fraud detection depends on accurate data — duplicates could create false alerts.





---

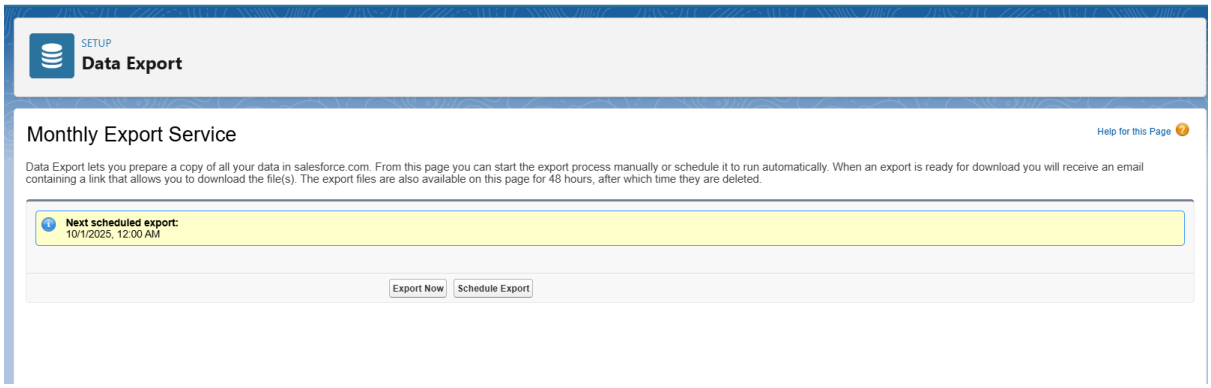
4. Data Export & Backup:

Purpose: Ensure regular backup of fraud data.

Steps:

- Setup → Data Export → Schedule Weekly Export.
- Export all custom objects: Transaction, Customer Profile, Fraud Investigation.
- Store CSV backups securely.

Why Important?
Prevents data loss during errors or deployment, ensures audit history of fraud investigations.



---

5. Change Sets:

Purpose:
Change Sets are normally used to move custom objects, fields, Apex classes, flows, and Lightning pages from Sandbox to Production without re-building.

Steps (real-world process):

1. In Sandbox Org → Setup → Outbound Change Sets → New Change Set.
2. Add components: Custom Objects, Apex Classes, Flows, Validation Rules, Approval Processes, Lightning Pages.
3. Upload Change Set to Target Org.
4. In Production Org → Inbound Change Sets → Deploy.

Note for Our Project:
Since we are working in a Developer Edition Org, Change Sets are not available.
Instead, we documented the process for completeness but used VS Code & SFDX to manage deployments in our project.

---

6. Unmanaged vs Managed Packages (Optional)

- **Unmanaged:** Share project with classmates → editable.
- **Managed:** Locked → AppExchange style.

Our Case: Use Unmanaged Package if sharing project. No AppExchange publishing needed.

---

7. ANT Migration Tool:

Purpose: Command-line deployment tool.

- Used in enterprise teams with CI/CD pipelines.
- Not required for this project.

---

8. VS Code & SFDX:

Purpose:
VS Code with Salesforce CLI (SFDX) gives a developer-friendly way to manage Salesforce metadata, automate deployments, and maintain version control with Git. Unlike Change Sets, this approach works in Developer Orgs and is preferred for real projects.

Steps Followed:

1. **Install Salesforce CLI (SFDX):**
    - Download from Salesforce CLI website.
    - Install it on your system.
2. **Install VS Code & Salesforce Extension Pack:**

- Install Visual Studio Code.
- In VS Code → Extensions Marketplace → Search Salesforce Extension Pack → Install.

3. **Authorize Dev Org in VS Code:**

- Open Command Palette in VS Code (Ctrl + Shift + P).
- Type: SFDX: Authorize an Org.
- Choose “Dev Hub” or “Sandbox/Developer Org” → Log in via browser → Allow access.

4. **Create/Connect Project:**

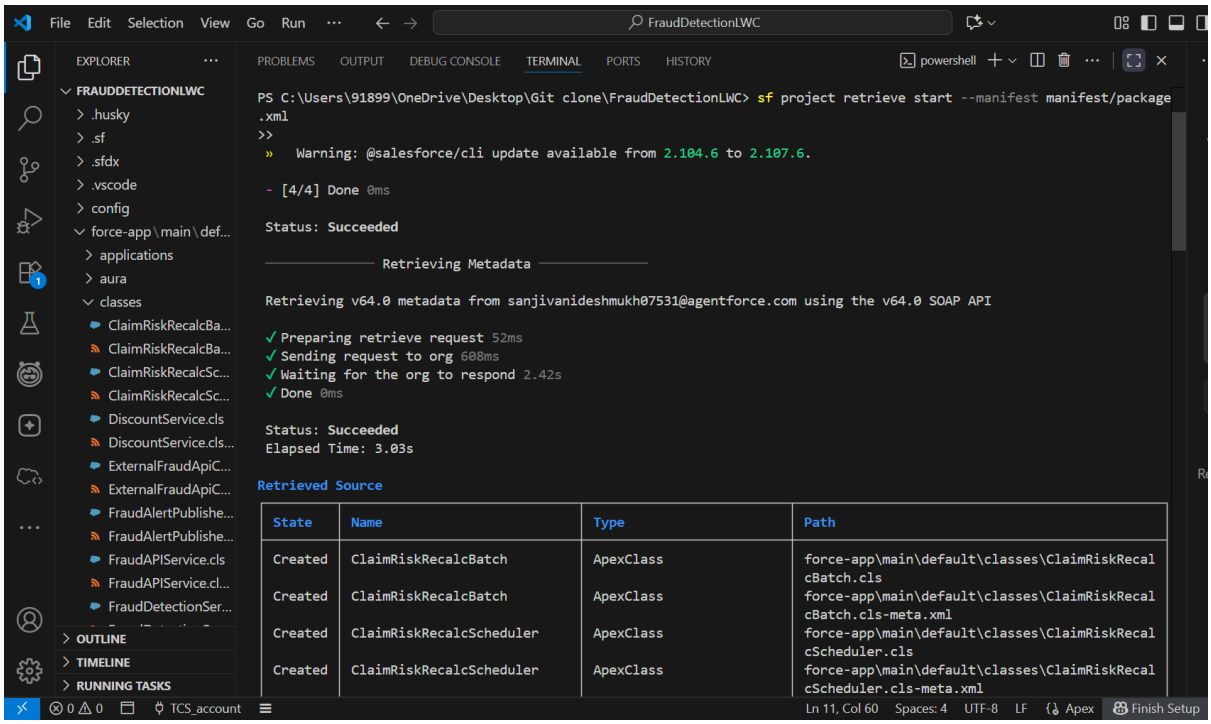
- In VS Code → Command Palette → SFDX: Create Project with Manifest.
- Select a folder for your project.
- This generates a manifest/package.xml file.

5. **Pull Metadata from Org:**

- In the terminal (inside your VS Code project folder), run:

sf project retrieve start --manifest manifest/package.xml

- This fetches all metadata defined in package.xml (e.g., custom objects, Apex classes, LWCs).



Phase 9: Reporting, Dashboards & Security Review

Goal:
Monitor insurance claims, detect high-risk/fraudulent claims, track investigation progress, analyze trends, and secure sensitive customer and fraud data.

---

1. Reports

Step 1: Go to Reports

1. In Salesforce, click the **App Launcher (⚡)**.
2. Search for **Reports** → Click **New Report**.

Step 2: Select Report Type

1. Choose the **Insurance Claim** report type.
    - If you have a **custom report type** (Claims + Fraud Investigations), you can use that too.
2. Click **Continue**.

Step 3: Add Filters

1. **Date Range:** Set to `All Time` or a relevant range (e.g., `Last 90 Days`).
2. **Fraud Score:** Keep all values for now (we'll calculate averages).
3. **Claim Status:** Optionally filter for `Approved` or `Closed` claims if you only want finalized ones.

Step 4: Add Columns

Make sure the following fields are included:

- **Claim Number**
- **Claim Type** (e.g., Medical, Vehicle, Property, etc.)
- **Claim Amount**
- **Fraud Score**
- **Fraud Flag (High Risk Yes/No)** → This is usually derived if `Fraud Score > 80`.

Step 5: Group by Claim Type

1. In the report builder, **drag Claim Type** into the **Group Rows** section.
    - This will organize all claims under their type.

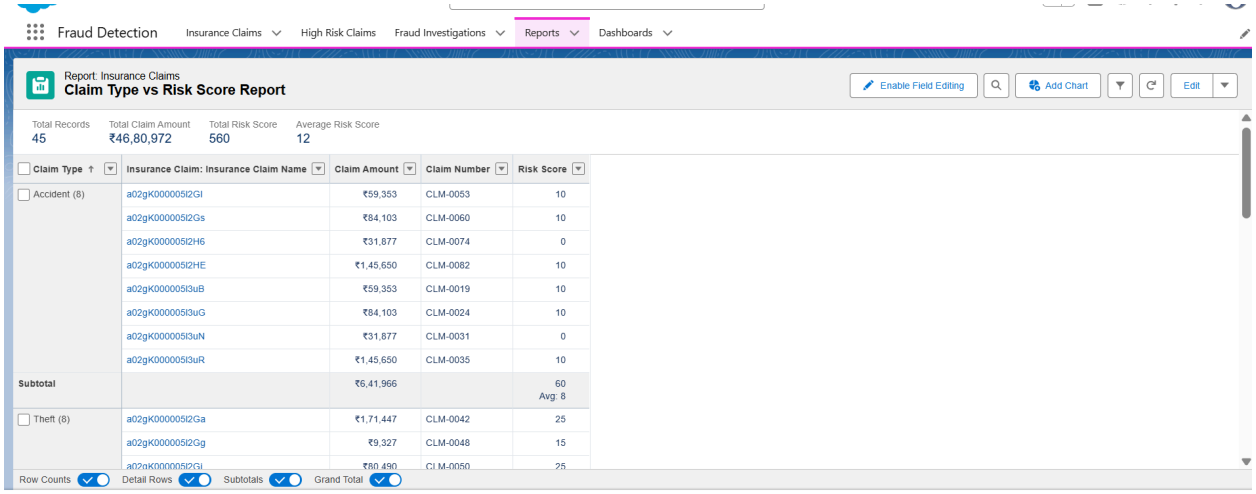
Step 6: Summarize Fraud Score

1. Hover over **Fraud Score column → Click ▼ → Summarize**.
2. Choose **Average**.

- This will show the average fraud score per claim type.

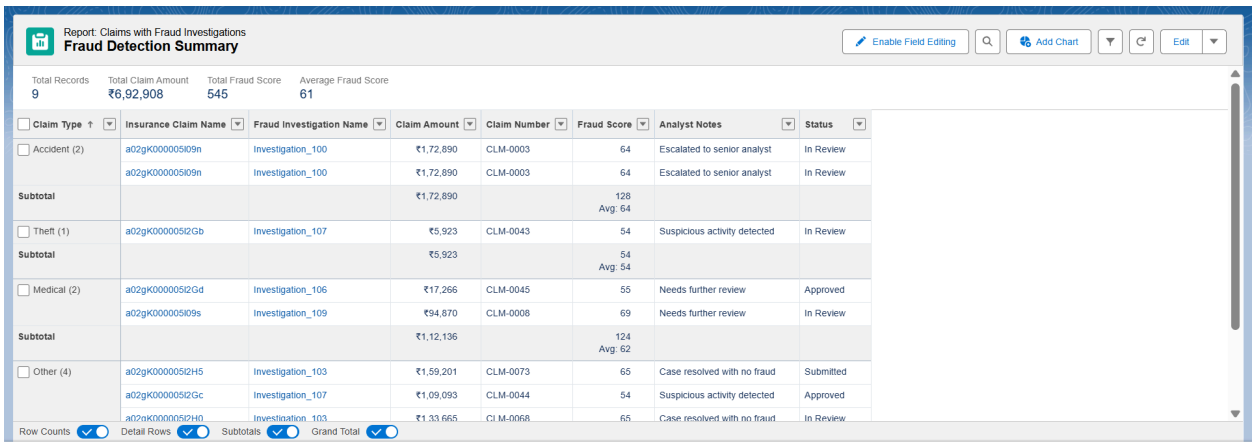
Step 7: Save the Report

1. Click **Save & Run**.
2. Report Name: `Claim Type vs Fraud Score Report`.
3. Folder: `Fraud Monitoring Reports`.



2. Custom Report Types

- **Object:** Insurance Claim + Fraud Investigation.
- **Fields:** Claim Number, Customer Name, Claim Amount, Fraud Score, Investigation Status, Analyst Notes.
- **Summary Reports:** Aggregate Fraud Scores by Claim Type, Region, or Analyst.
- **Purpose:** Enable detailed reporting and dashboards to analyze high-risk claims, trends, and analyst performance.



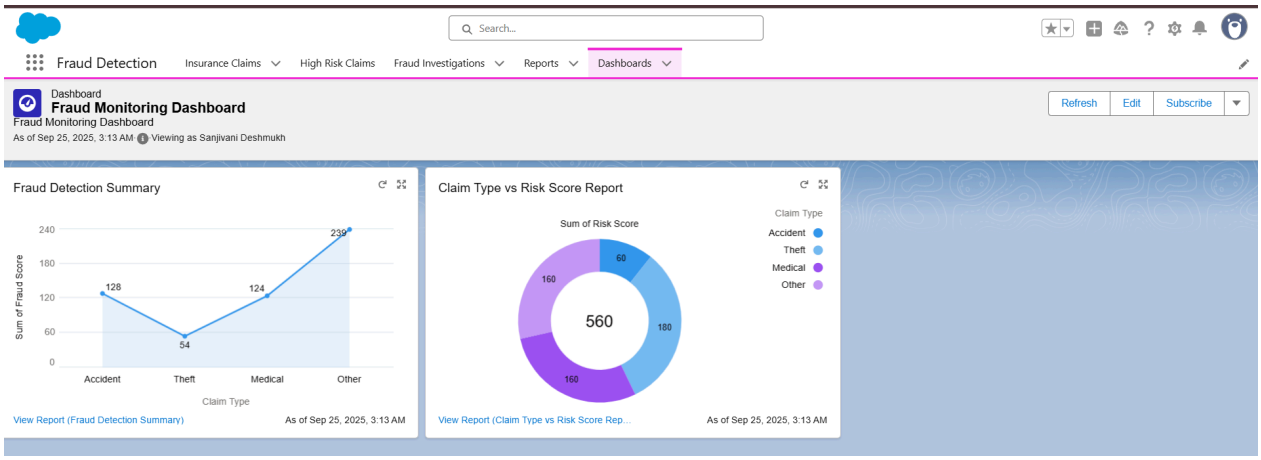
3. Dashboards

Visualize fraud patterns and business impact:

- **Fraud Monitoring Dashboard:** Donut chart showing Claim Type vs Risk Score Report and Line chart showing Fraud Detection Summary.

Steps:

1. Go to **Dashboards → New Dashboard**.
2. Add components using custom reports.
3. Set chart types and groupings.
4. Apply filters (Time Frame, Claim Type, Analyst).
5. Save and refresh to display live data.



4. Sharing Settings

- **Sharing Settings:** Claims private, Fraud Investigations inherit claim visibility.

Customer Profile	Public Read Only	Private	✓
Fraud Investigation	Private	Private	✓
Insurance Claim	Private	Private	✓

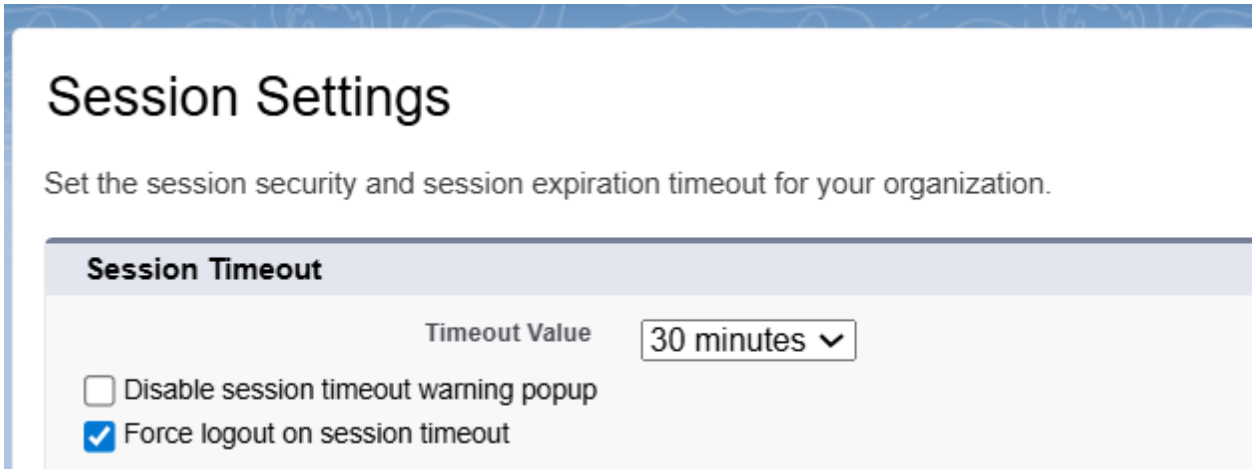
5. Field Level Security

- Hide sensitive fields like Customer ID Proof and Bank Account Details from junior analysts.
- Fraud Score visible only to analysts and managers.

Steps: Setup → Object Manager → Field-Level Security → adjust visibility per profile.

6. Session Settings

- **Session Timeout:** 30 minutes of inactivity → auto logout.



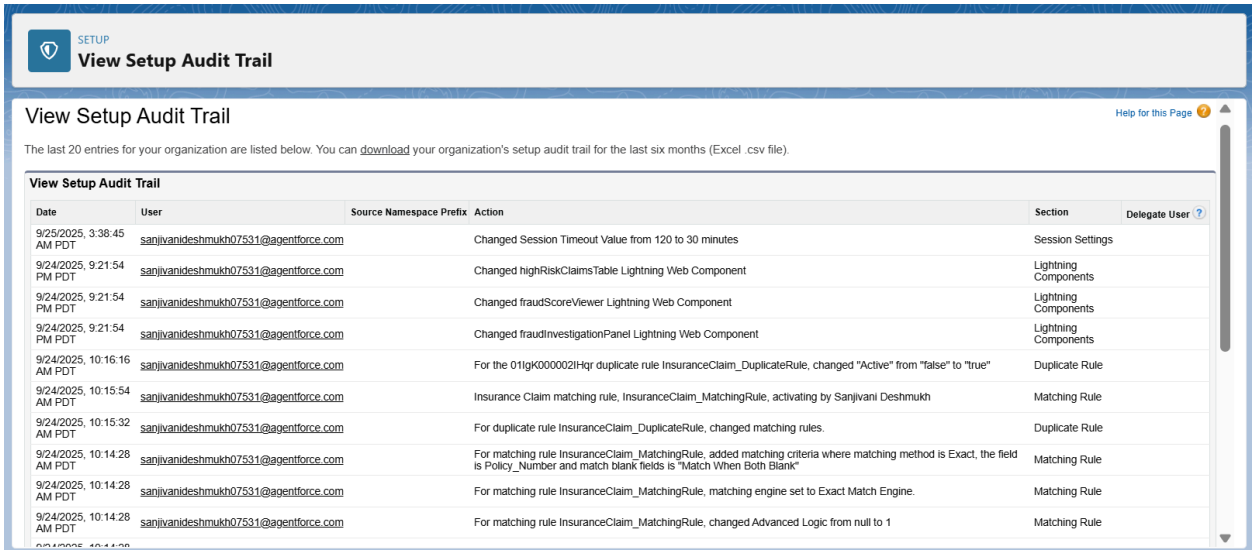
7. Login IP Ranges

Login IP Ranges: Restrict access to office VPN or secure networks.

Steps: Setup → Profiles (Claims Adjuster Profile)→ Session Settings & Login IP Ranges.

8. Audit Trail

- Track who created, modified, or deleted claims and investigations.
- Enables **compliance and accountability**.
- Steps: Setup → Audit Trail / Field History Tracking → enable for key objects.


SETUP
View Setup Audit Trail
View Setup Audit Trail
Help for this Page
The last 20 entries for your organization are listed below. You can download your organization's setup audit trail for the last six months (Excel .csv file).
View Setup Audit Trail
Date
User
Source Namespace Prefix
Action
Section
Delegate User
9/25/2025, 3:38:45 AM PDT
sanjivanideshmukh07531@agentforce.com
Changed Session Timeout Value from 120 to 30 minutes
Session Settings
9/24/2025, 9:21:54 PM PDT
sanjivanideshmukh07531@agentforce.com
Changed highRiskClaimsTable Lightning Web Component
Lightning Components
9/24/2025, 9:21:54 PM PDT
sanjivanideshmukh07531@agentforce.com
Changed fraudScoreViewer Lightning Web Component
Lightning Components
9/24/2025, 9:21:54 PM PDT
sanjivanideshmukh07531@agentforce.com
Changed fraudInvestigationPanel Lightning Web Component
Lightning Components
9/24/2025, 10:16:16 AM PDT
sanjivanideshmukh07531@agentforce.com
For the 01IgK000002IHqr duplicate rule InsuranceClaim_DuplicateRule, changed "Active" from "false" to "true"
Duplicate Rule
9/24/2025, 10:15:54 AM PDT
sanjivanideshmukh07531@agentforce.com
Insurance Claim matching rule, InsuranceClaim_MatchingRule, activating by Sanjivani Deshmukh
Matching Rule
9/24/2025, 10:15:32 AM PDT
sanjivanideshmukh07531@agentforce.com
For duplicate rule InsuranceClaim_DuplicateRule, changed matching rules.
Duplicate Rule
9/24/2025, 10:14:28 AM PDT
sanjivanideshmukh07531@agentforce.com
For matching rule InsuranceClaim_MatchingRule, added matching criteria where matching method is Exact, the field is Policy_Number and match blank fields is "Match When Both Blank"
Matching Rule
9/24/2025, 10:14:28 AM PDT
sanjivanideshmukh07531@agentforce.com
For matching rule InsuranceClaim_MatchingRule, matching engine set to Exact Match Engine.
Matching Rule
9/24/2025, 10:14:28 AM PDT
sanjivanideshmukh07531@agentforce.com
For matching rule InsuranceClaim_MatchingRule, changed Advanced Logic from null to 1
Matching Rule